# F02 — Architecture method, views, and decision records

**Complete learning and senior Solution Architect interview reference**

**Authoritative syllabus:** `solution_architect_interview_topic_map_v3.md`, revision v3, 2026-10-24.

**Scope:** F02 only — **C019, C020, C021 (Core)** and **C022 (Depth)**.

**Prepared and primary-reference links checked:** 2026-10-24.

**Status:** Reference guide and uncompleted assessment; no learner performance is assumed.

> **Read this first.** This is a self-study reference document, not an interactive lesson or a completed examination. Work through the short lessons in order. Do the exercises before reading the separate answer key. Every diagram and business example below is **hypothetical**, not an American Express system description. There is no claim that the actual job posting, internal architecture, technology stack, design-review governance, or interview format has been verified.

REFERENCE MAP

# Contents

Use the page references to move between foundations, twelve progressive lessons, assessments, the answer key, and the final audit.

## FOUNDATIONS & CROSSWALK

1. Reading key, scope, and study method 3
2. Prerequisite check — F01 and basic software language 3
3. Coverage matrix and assessment crosswalk 6

## LESSONS 01–12

4. L01 — Architecture method, audiences, and viewpoints [C019 Core] 8
5. L02 — Context view: who and what surrounds us? [C019 Core] 10
6. L03 — Component view: responsibilities inside the boundary [C019 Core] 12
7. L04 — Data-flow view: what information crosses which boundary? [C019 Core] 14
8. L05 — Sequence view: order, responses, and uncertain outcomes [C019 Core] 16
9. L06 — Deployment view: where software runs and what can fail [C019 Core] 19
10. L07 — Choose views by concern; check consistency across all five [C019 Core] 21
11. L08 — Compare realistic options with explicit criteria [C020 Core] 23
12. L09 — Write an architecture decision record with consequences and triggers [C020 Core] 26
13. L10 — Trace requirements to decisions, tests, risks, dependencies, and owners [C021 Core] 29
14. L11 — Formal architecture review: people, agenda, evidence [C022 Depth] 33
15. L12 — Decide whether the design is ready to implement [C022 Depth] 36

## PRACTICE, ANSWERS & EVIDENCE

16. Assessment exercises — do these before opening the answer key 40
17. Separate answer key — sample solutions, not learner results 43
18. Assessment rubric, scorecard and completion rules 51
19. Primary-source register and verification limits 54
20. Final completeness audit — document coverage versus learner mastery 56

# 1. Reading key, scope, and study method

Every material claim is identified by one of four labels:

- **Source requirement [SR]** — the binding F02 syllabus in the supplied v3 topic map. It determines *what* must be learned and assessed.
- **Verified external fact [VF]** — the linked primary issuer, standards body, author, or technical organization supports the stated fact. Exact sources appear in §19.
- **Hypothetical practice assumption [HA]** — an invented case detail for learning; never repeat it as company fact.
- **Recommendation [REC]** — proposed practice or teaching convention, not a mandatory industry rule.

**Learning order [SR]:** Understand → Explain → Demonstrate → Practice → Evaluate → Memorize. For each checklist item, the six required parts are purpose/prerequisites, terms, worked example, guided exercise, failure/trade-off, and interview phrasing with a follow-up. Here they appear under each lesson. Model exercise answers are deliberately held until **§16**.

**Small-step method [REC]:** (1) read one lesson; (2) redraw the example from memory; (3) answer its exercise without looking; (4) explain the trade-off aloud; (5) use the Interview Phrasing only after understanding it; (6) score your result and revisit gaps. Do not mark a checklist ID complete based on reading alone.

### What is in and out of scope

**In scope [SR]:** five architecture views for audiences; option comparisons; architecture decision records and reconsideration triggers; requirement-to-decision/test/risk/dependency/owner traceability; a formal design review and implementation-readiness decision (Depth); the prescribed running-case exercise and senior decision test.

**Not taught as independent F02 subjects [SR]:** detailed payment rails or settlement rules (D01/D02), microservice design patterns (T01), distributed delivery guarantees (T02), full API design (T03), full threat modeling (T06), production resilience engineering (T07), cloud service selection (T10), and enterprise governance policies (E02). A few terms appear only to make an F02 diagram meaningful. They are marked as placeholders requiring specialists or later cards. Formal notation mastery and diagramming tools are **not** a goal.

# 2. Prerequisite check — F01 and basic software language

**F02 prerequisite [SR]: F01.** If F01 is unfamiliar, use this short bridge before Lesson 1. It does **not** replace passing F01.

| Term | Plain-English meaning | Tiny example |
|---|---|---|
| Software system | Programs and stored data working together for a purpose | A commercial spending platform |
| User/actor | Person or outside system interacting with the system | Employee, risk-check system |
| Business goal | Desired result for a business | Employees can make policy-compliant purchases |
| Business capability | Something the business must be able to do | Enforce spending rules |
| Journey | Steps a user or business action follows | Submit → evaluate → show status |
| Stakeholder | Someone affected by or responsible for a choice | Product, engineering, finance, risk, operations |
| Requirement | What the solution must do or achieve | Reject a purchase that breaks a rule |
| Functional requirement | Behavior the system must provide | Record a purchase request |
| Quality attribute | Measurable quality such as security or timeliness | Report freshness is within an agreed bound |
| Constraint | A condition the design cannot ignore | Existing risk service must be used |
| Assumption | An unverified detail used temporarily | Reporting may be delayed |
| Open question | Important missing information | Who owns final transaction status? |
| Application programming interface (API) | Contract specifying how software communicates | `submitPurchase(request)` |
| Data store | Place data is kept beyond one request | Transaction-record store |
| Owner | Person/team accountable for an artifact or outcome | Transaction-state team |
| Test | Check that observed behavior meets an expectation | Send a request above a limit |
| Risk | An uncertain event that could cause harm | A transaction loses its audit record |
| Dependency | Something this design relies on | Risk service, payment interface |
| Trade-off | A choice that improves one quality at the cost of another | Faster delivery vs. more operational protection |

**F01 readiness questions (PR-1 to PR-4):** PR-1. Name three user roles. PR-2. Write one functional requirement. PR-3. Write one measurable quality scenario with actor, situation, expected response, and measure. PR-4. Name an assumption and an owner who can confirm it. **Expected outcome:** all four are answered clearly before Lesson 1. If not, return to F01. Suggested answers are in §16.

## One controlled running case [HA]

The topic map prescribes a **hypothetical** commercial platform: administrators set spending rules; employees initiate purchases; finance teams view and reconcile transactions; the platform connects to identity, risk, payment, and reporting capabilities.

To make F02 diagrams internally consistent, this guide temporarily assumes:

| ID | Hypothetical assumption; not a company fact | Validation owner / missing evidence |
|---|---|---|
| H01 | A single **Spending Platform** is our system of interest. Identity, Risk, Payment, and Reporting are **external interfaces** for this exercise. | Product and integration architects verify actual boundaries |
| H02 | Employees submit a **purchase request** and get a status. A request is **not** equivalent to a settled financial transaction. | Product + finance confirm state terminology |
| H03 | Administrators define a policy; an application component checks it before invoking Risk and Payment interfaces. | Business and risk owners confirm control order |
| H04 | A **Transaction State Repository** is proposed as the authoritative record of the platform's *request-processing state*, **not** necessarily of final financial settlement. | Domain/data owners must agree |
| H05 | Finance reporting can be **delayed** if an agreed freshness limit is met; that limit has **not** been approved. | Finance product owner confirms acceptable delay |
| H06 | Two integration options are plausible: a synchronous reporting call, or a durable asynchronous handoff. | Engineering and operations validate feasibility |
| H07 | For one teaching calculation only, the draft reporting freshness target is **5 minutes**; this is invented and unapproved. | Finance must approve or change it |
| H08 | The draft implementation places application processing in one private runtime zone with a separate database node. Network, region, and failure-domain details are unverified. | Platform/security/operations validate deployment |

| ID | Hypothetical assumption; not a company fact | Validation owner / missing evidence |
|---|---|---|
| H09 | A purchase uses a stable **request ID** such as `REQ-17` for tracking; exact retry and duplication behavior remain unresolved. | Payment/integration owners verify before implementation |

**Boundary rule:** A box shown as one component does **not** imply a microservice, a separate team, or a separate machine. A component is a logical part until a deployment choice is made. **[REC]**

**Diagram reading key:** `[Name]` = role, system, component, or store; `-->` = direction of a described interaction or data movement; `..>` = later/asynchronous or proposed dependency; `X` = unavailable; `?` = undecided. Direction alone never guarantees success, delivery, security, or ownership. Every diagram should have a **title, audience, scope, assumptions, legend, and named open questions**. **[REC]**

# 3. Coverage matrix and assessment crosswalk

| Checklist ID / level | Exact required capability [SR] | Explicit lesson inventory | Evidence to create | Assessment |
|---|---|---|---|---|
| **C019 · Core** | Produce **context, component, data-flow, sequence, and deployment** views for different audiences. | L01 (method/viewpoint), **L02 context**, **L03 component**, **L04 data-flow**, **L05 sequence**, **L06 deployment**, **L07 view selection and consistency** | Five separately labeled diagrams; audience/question/assumptions for each; normal and failure sequence; cross-view check | E01–E07; Q01–Q07; Capstone K1–K2; Senior S1 |
| **C020 · Core** | Compare options using explicit criteria; write architecture decision records with consequences and revisit triggers. | **L08 comparison**, **L09 decision record** | Criteria and evidence table; 2+ options; recorded decision, alternatives, consequences, trigger, owner | E08–E09; Q08–Q09; K3; S2 |
| **C021 · Core** | Trace requirements to design decisions, tests, risks, dependencies, and owners. | **L10 traceability** | Bidirectional matrix; intentional gap detection; owner + evidence of each test | E10; Q10; K4; S3 |
| **C022 · Depth** | Practice a formal architecture review and evaluate whether a design is ready to implement. | **L11 review**, **L12 readiness decision** | Review pack, reviewer roles, recorded findings, dispositions, conditions and sign-off | E11–E12; Q11–Q12; K5; S4 |

**Separation rule:** Each of the five C019 views is an individually taught and assessed deliverable. C020, C021, and C022 also have their own explain/apply/defend gates. The depth item is visibly assessed but is **not** quietly promoted to Core.

# 4. L01 — Architecture method, audiences, and viewpoints [C019 Core]

## 1) Purpose and prerequisites

**Why it exists:** Complex systems cannot be understood from one picture. An architecture description is a set of related explanations of a system. It helps different people answer different questions before teams commit to implementation. Prerequisite: a stakeholder, a requirement, and a system boundary (see §2).

**Verified external fact [VF]:** ISO/IEC/IEEE 42010:2022 addresses the **structure and expression** of an architecture description, including viewpoints and model kinds; its published abstract **does not prescribe a particular diagramming process, notation, or tool**. See [S1]. The creator-maintained C4 guidance describes diagrams at different levels of detail for different audiences [S2].

## 2) Terms and fundamentals

- **Architecture:** Significant structure, choices, and relationships that affect a system's qualities and its evolution. Here this is a teaching definition **[REC]**, not a verbatim quote from ISO.
- **Architecture description:** A recorded representation of architecture; it may contain diagrams, text, decisions, and evidence.
- **Concern:** A stakeholder's question, for example “Can Finance trust the transaction status?”
- **Viewpoint:** A rule for what a view should explain and to whom; for example an operational viewpoint focuses on runtimes and failure boundaries.
- **View:** A concrete picture or table made for a particular viewpoint; one diagram can support multiple concerns if it stays legible.
- **Model:** A selective simplification of the real world. Omitted details must not be mistaken for absent behavior.
- **System boundary:** What is inside our responsibility versus an external dependency.
- **Logical vs. physical:** Logical = responsibility and relationships; physical = where something runs.

**Workflow [REC]:** (1) start with F01 goals and risks; (2) name the audience and its questions; (3) choose the smallest suitable view; (4) state scope and assumptions; (5) draw boxes and labeled relationships; (6) review for consistency; (7) attach decisions, tests, risks, and owners.

## 3) Worked example — who needs which answer?

| Stakeholder | Main question | View to start with | Reason |
|---|---|---|---|
| Product manager | Who uses the platform and what other systems does it rely on? | Context | Shows people and boundaries |
| Engineer | Which part owns policy checking and request state? | Component | Shows internal responsibilities |
| Data/privacy reviewer | Where do request details travel and get stored? | Data-flow | Shows data, destinations, trust boundaries |
| Business tester | What happens when Risk declines a request? | Sequence | Shows order and responses |
| Operations | Which runtime and database could fail? | Deployment | Shows runtime locations and network dependencies |

**Walkthrough:** The *same* Spending Platform appears in each row. A context view deliberately omits component classes. A deployment view should show the same logical application at runtime, not invent unrelated components. You do not need five diagrams for every small change in real work, but F02 requires practicing all five.

## 4) Guided exercise — E01

**Goal:** Pick the right audience/view. **Steps:** (a) For “Where does the transaction state live?” identify the stakeholder concern; (b) name the first diagram; (c) state one question that needs another diagram. **Expected:** view selection plus a justified follow-up. **Hint:** ownership and physical location are different concerns.

## 5) Failure, trade-off, senior reasoning

**Failure:** One picture mixes external vendors, source-code classes, server names, and payment states. **Detection:** Readers cannot answer one question without decoding unrelated detail. **Repair:** Split logical, behavioral, and deployment views, then reconcile shared names. **Trade-off:** More views improve clarity but increase maintenance cost. **Senior reasoning:** Maintain only the views that answer material questions, but make the required five explicitly for F02 practice.

## 6) Interview Phrasing

> “I start with the stakeholder's question, then choose a view at the right level of detail. I keep the system boundary and terminology consistent across the views, and I connect the diagrams to the decisions and tests they support.”

**Plain English:** First ask who needs the answer. Draw only the information they need. Check that the five drawings describe the same system. **Follow-up:** “How do you prevent two diagrams from contradicting each other?”

# 5. L02 — Context view: who and what surrounds us? [C019 Core]

## 1) Purpose and prerequisites

A **context diagram** answers: “What is the system, who uses it, and which external systems does it communicate with?” It is useful when agreeing scope, accountability, and external dependencies. Prerequisite: be able to name users and distinguish inside from outside (§2). **[VF]** The C4 system-context guidance describes a single system in scope surrounded by people and other systems; technical details are intentionally secondary [S3].

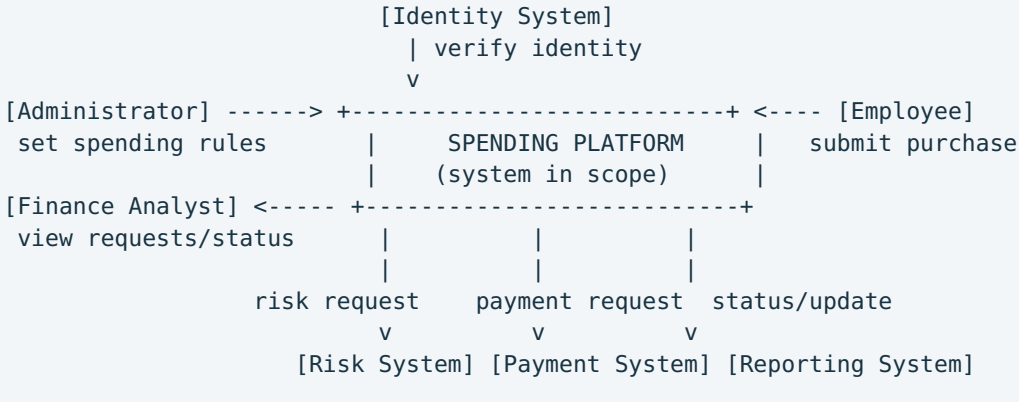
## 2) Terms and fundamentals

- **System of interest:** The system currently being designed or reviewed.
- **External system:** Something connected to, but outside, the selected boundary.
- **Actor:** User or other entity interacting with the system.
- **Interface:** Defined interaction between systems, such as a request or message; not a promise about its internal implementation.
- **Dependency:** An external behavior needed for part of the journey.

**How to build [REC]:** (1) label system and boundary; (2) add people; (3) add only directly related external systems; (4) label each line with a verb and relevant business data; (5) remove internal detail; (6) ask an owner to confirm each interface.

## 3) Worked example — Context-01 [HA]

**Audience:** product/business sponsor and integration leads. **Question:** “What is in scope, and who must be involved?”




**Read it step by step:**

1. The central box is the *only* system in scope for this picture.

2. Administrator, Employee, and Finance Analyst are roles, not software services.
3. Identity, Risk, Payment, and Reporting are dependencies, **hypothetically** outside the boundary (H01).
4. Arrows are labeled by business meaning, not by guessed protocols.
5. This view does **not** say which internal module stores transaction state or where it runs. That is correct for a context view; those answers require other views.

**Assumptions:** H01–H03. **Open questions:** Who owns Payment? Is Reporting internal in the real organization? Who is accountable for failed request recovery?

## 4) Guided exercise — E02

**Goal:** Correct the context scope. **Steps:** Draw a new context view that includes an external Finance Export system and an auditor role. Label two flows, mark boundaries, and explain whether a “Policy Validator” source-code class belongs here. **Expected:** one system-of-interest box, distinct actors/dependencies, justified omission of code classes. **Hint:** context asks *who*, not *which function*.

## 5) Failure, trade-off, senior reasoning

**Failure:** Draw Reporting inside the boundary in one document and outside it in another. **Detect:** names and ownership tables disagree. **Fix:** mark external/inside status as **unconfirmed**, identify boundary owner, update all diagrams after a decision. **Trade-off:** A wider boundary reduces external-looking arrows but hides integration accountability. **Senior reasoning:** Determine organizational and runtime responsibility from evidence; don't equate a diagram box with legal or operational ownership.

## 6) Interview Phrasing

> “My context view establishes the system of interest, its users, and external integrations. I label interactions in business terms and use it to confirm boundaries with stakeholders before decomposing the design.”

**Plain English:** This is the map of the system's neighbors. **Follow-up:** “What would you ask if the payments team challenges your boundary?”

# 6. L03 — Component view: responsibilities inside the boundary [C019 Core]

## 1) Purpose and prerequisites

A **component view** answers: “Which logical parts do work, what does each own, and how do they collaborate?” Prerequisite: know the context boundary (L02). **[VF]** In strict C4 terminology, a **component diagram** zooms into a single **container** (an application or data store that runs or stores data). See the creator's definition [S4]. Our example is a simplified *logical component view* of an assumed application container; it is **not** claiming that a C4 component is a separate deployable microservice.

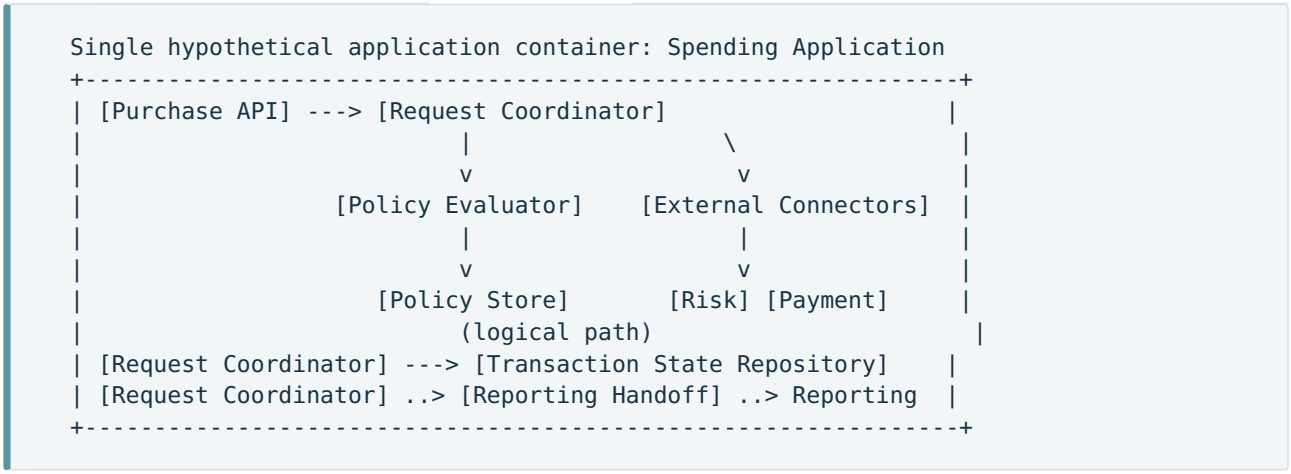
## 2) Terms and fundamentals

- **Component:** A cohesive logical unit that does one set of responsibilities.
- **Responsibility:** Work a unit must carry out and answer for.
- **Cohesion:** Related tasks are grouped together.
- **Coupling:** How much one part depends on another's implementation or contract.
- **Container (C4 sense):** A separately running application or data store; **not necessarily** a Docker container.
- **Authoritative source / system of record:** The explicitly designated owner of a particular fact or state; different facts can have different authoritative sources.
- **State:** Information describing the current known condition of a request, such as `PENDING_REVIEW`.

**How it works [REC]:** Start from one business capability; name components by responsibility, label calls, state who writes important data, and review which components may change independently. Never infer that separate boxes require separate servers or teams.

## 3) Worked example — Component-01 [HA]

**Audience:** application engineers and data owner. **Question:** “Which component owns request-processing state?”




**Read it step by step:**

1. `Purchase API` receives a request; **API** means a software communication contract.
2. `Request Coordinator` orchestrates the steps. It does not silently own all underlying facts.
3. `Policy Evaluator` reads policy rules; `Policy Store` owns saved policy data in this draft.
4. `External Connectors` send and receive requests to Risk and Payment systems (H01).
5. `Transaction State Repository` is the **proposed** authoritative source for the platform's processing state (H04). It is not evidence of financial settlement.
6. `Reporting Handoff` is a boundary/adapter to reporting; the dotted arrow means its mechanism is undecided between the two H06 options.

**Important consistency note:** The diagram is conceptual. A repository may be a database access component backed by a separate data store, as shown in L06. The “Policy Store” and “Transaction State Repository” may later be logical schemas within the same database; F02 does not choose that physical arrangement.

## 4) Guided exercise — E03

**Goal:** Make ownership explicit. **Steps:** (a) Add an `Audit Writer` responsibility for changes to spending rules; (b) specify the information it records; (c) decide whether it is a new deployable service or only a component; (d) add the *request-processing state owner* as a one-line annotation. **Expected:** named logical responsibilities without unsupported physical claims. **Hint:** audit history differs from current state.

## 5) Failure, trade-off, senior reasoning

**Failure:** `Request Coordinator` and Reporting both treat themselves as the owner of transaction state. **Detect:** two boxes can independently overwrite a status without an authority rule. **Fix:** designate an authoritative owner per state, allowed read models, update contract, and error path; confirm with data/finance. **Trade-off:** More components clarify responsibilities but add interfaces and maintenance. **Senior reasoning:** Choose boundaries by clarity, change, and risk; don't propose microservices merely because boxes look separate. Detailed service decomposition is T01, not F02.

### 6) Interview Phrasing

> "I use a component view to assign responsibilities and interfaces. I make the owner of important state explicit, because a neat diagram without ownership leaves correctness and incident response unclear."

**Plain English:** The picture must say which part is accountable. **Follow-up:** "What if finance disagrees with your proposed state owner?"

# 7. L04 — Data-flow view: what information crosses which boundary? [C019 Core]

### 1) Purpose and prerequisites

A **data-flow diagram (DFD)** answers: "Where does information originate, go, change, and remain stored?" It helps identify missing data ownership, sensitive information, and third-party boundaries. Prerequisites: context and components (L02–L03). **[VF]** Microsoft Security Engineering recommends a high-level DFD as an architecture overview and discusses trust boundaries and labels for data types and protocols [S5]. We use a light ASCII form rather than claiming formal DFD notation conformance.

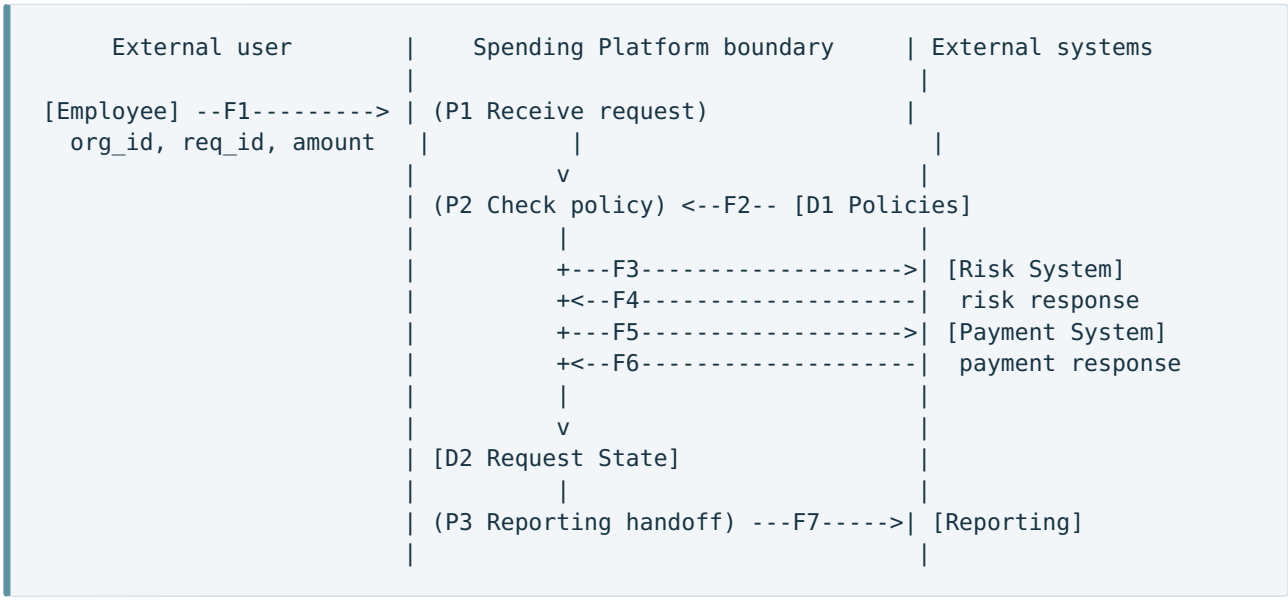
### 2) Terms and fundamentals

- **Data item:** One meaningful piece of information (e.g., request ID, amount, organization ID).
- **Process:** Step that receives or transforms data.
- **Data store:** Place where information persists.
- **Data flow:** Movement of labeled data from a source to a destination.
- **Trust boundary:** Crossing between zones, systems, or responsibilities with different trust assumptions. It signals the need to review controls; the line alone does not enforce security.
- **Data owner:** Accountable party for definitions, changes, access, and correctness of specified data.
- **Data minimization:** Sending only necessary information for a task.

**Method [REC]:** Number important flows. For each, label source, receiver, fields, purpose, store, classification/permission question, owner, and whether it crosses a boundary. **Do not** confuse an arrow with a documented authorization or data-sharing approval.

## 3) Worked example — DFD-01 [HA]

**Audience:** data, privacy, and security reviewers. **Question:** “Who can receive purchase details and where is request status persisted?”




### Read it step by step:

1. **F1** contains a request identifier (`req_id`), organization identifier (`org_id`), and amount; these are invented field names, not a real contract.
2. **D1** is policy data; **D2** is the platform's *processing* state store. State ownership remains conditional on H04.
3. **F3–F6** cross to external Risk and Payment services. A “response” might say pending, approved for the immediate action, rejected, or unknown; no final settlement is implied.
4. **F7** exports only the minimum reporting fields that Finance needs. Whether F7 is immediate or durable is **open** (H06).
5. Privacy/security reviews must decide identity handling, authorization, sensitive-data classification, retention, and access control; the drawing does not validate these details.

### Data ownership mini-table [HA]:

| Data item | Proposed authoritative owner | Consumers | Open concern |
|---|---|---|---|
| Spending policy | Policy domain owner | Policy Evaluator | Who authorizes edits? |
| Platform request-processing state | Transaction state owner (unconfirmed) | Employee view, Reporting | Which event changes state? |
| External payment outcome | Payment system/domain owner (unconfirmed) | Request Coordinator | Which external response is definitive? |
| Reporting snapshot | Reporting owner | Finance users | How stale may it be? |

### 4) Guided exercise — E04

**Goal:** Make the data flow reviewable. **Steps:** (a) mark every flow across the Spending Platform boundary; (b) flag F7 as potentially sensitive; (c) propose only three fields Finance requires; (d) name one data owner and one unverified security question. **Expected:** flow inventory plus explicit unknowns. **Hint:** “Send everything for debugging” is not a valid minimization strategy.

### 5) Failure, trade-off, senior reasoning

**Failure:** Reporting receives a full employee profile when it needs only organizational totals and request status. **Detect:** compare fields against business purpose and approved access. **Fix:** remove unnecessary fields, review contracts and retention, assign data-classification approval. **Trade-off:** Richer data eases analysis but increases exposure and ownership obligations. **Senior reasoning:** Trace authoritative facts and trust boundaries without pretending to have finished a full T06 threat model. A DFD is **not** a chronological sequence: F7's placement does not prove it always happens after a successful write.

### 6) Interview Phrasing

> “I add a data-flow view when correctness or sensitive information matters. I label the data crossing each boundary, the authoritative owner, and any unanswered access or retention questions.”

**Plain English:** Show where information travels, who controls it, and what is still unknown. **Follow-up:** “How would you respond if Reporting asks for all raw transaction fields?”

---

# 8. L05 — Sequence view: order, responses, and uncertain outcomes [C019 Core]

### 1) Purpose and prerequisites

A **sequence diagram** answers: “In what order do participants interact, and what happens when one step does not complete?” Prerequisites: request/response, components, and the difference between processing state and final financial state. **[VF]** The Object Management Group publishes Unified Modeling Language (UML) as a specification [S6]; the creator's C4 dynamic guidance also describes runtime collaboration and a sequence-style representation [S7]. **Our ASCII diagram is UML-inspired, not a full UML-conformant model.**

### 2) Terms and fundamentals

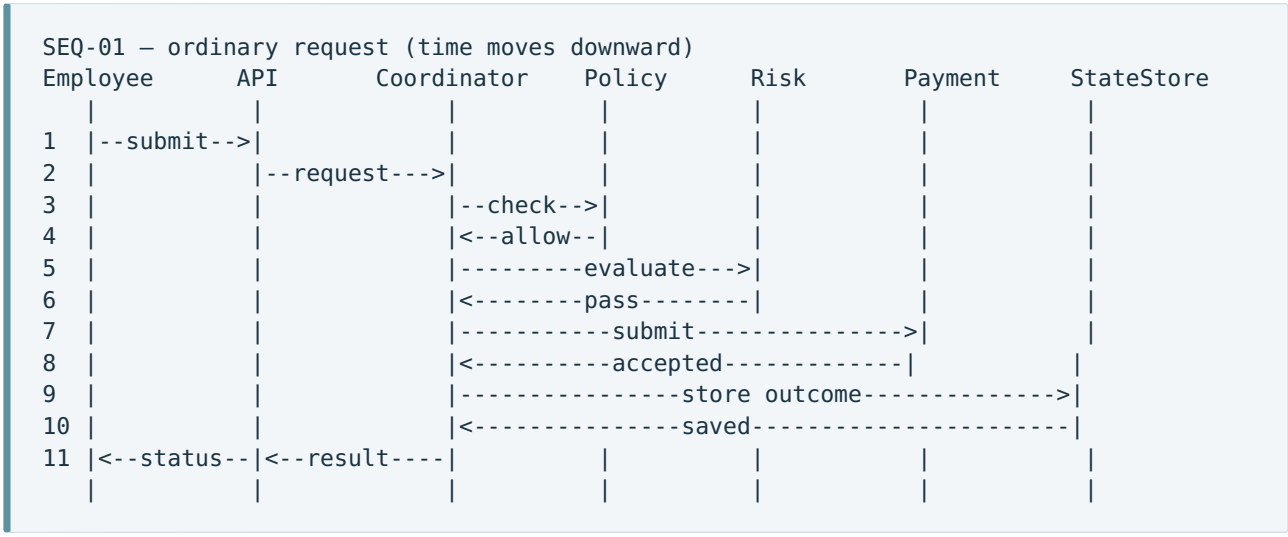
- **Participant / lifeline:** A person, component, or system involved in an interaction. Vertical placement represents passage of time, top to bottom.

- **Call/message:** A request or notification from one participant to another.
- **Response:** Information returned after a call; absence of a response may leave the result unknown.
- **Synchronous:** Caller waits for a response before moving forward.
- **Asynchronous:** The sender can proceed without waiting for the receiver to finish; delivery semantics must be defined separately.
- **Timeout:** A wait ends before a response is received. It does **not** prove the action failed.
- **Alternate path:** What happens after a different result, such as a policy decline or unknown outcome.

**Workflow [REC]:** Choose one user journey; name participants; show the normal path in order; add rejection and timeout branches; label stored states; identify who is responsible for uncertain outcomes. Avoid making settlement claims from a request acknowledgment.

## 3) Worked example — SEQ-01 normal, then SEQ-02 failure [HA]

**Audience:** engineers, QA (quality assurance) testers, and risk/domain leads. **Question:** “What will an employee see, and what do we know after a payment timeout?”




**Walkthrough by numbered line:** 1–2 receive and pass request `REQ-17`; 3–4 check policy; 5–6 ask Risk; 7–8 ask Payment; 9–10 save the platform's *known response*; 11 return that status. An external `accepted` response is **not final financial settlement**. The handoff to Reporting is omitted **from this timing view only** and documented in DFD-01 and ADR-01; an implementation review must verify the handoff mechanism.

```
SEQ-02 — timeout after external Payment may have acted
Employee     Coordinator             Payment                 StateStore
   |              |                     |                        |
   |--REQ-17----->|---submit REQ-17---->|                        |
   |              |       X timeout     | (may have processed)   |
   |              |---record UNKNOWN-------------------------->|
   |<--PENDING----|                     |                        |
   |              |----query status?--->|  (ability NOT verified) |
   |              |<---resolved or ?----|                        |
   |              |----update if evidenced--------------------->|
```

**Critical nuance:** Persisting `UNKNOWN` can itself fail. The sequence above shows an *intended* handling path, not proof it is durable. The proposed design **must not** blindly resubmit a potentially successful financial action. The system needs an approved reconciliation/resolution contract from the Payment owner; detailed safe retry logic belongs to T02.

## 4) Guided exercise — E05

**Goal:** Explain failure honestly. **Steps:** (a) add a policy-decline branch to SEQ-01; (b) add a failure where the StateStore write fails after Payment answers; (c) name the customer-visible status and what the system still does not know. **Expected:** consistent order and no false "failed payment" claim. **Hint:** distinguish "response not saved" from "payment did not happen."

## 5) Failure, trade-off, senior reasoning

**Failure:** A payment call times out; the client immediately retries with a new ID; a duplicate action may occur. **Detection:** correlate client request IDs and provider evidence; trace unresolved records. **Recovery:** preserve uncertainty, use approved query/reconciliation behavior, assign incident owner. **Trade-off:** Waiting for definitive confirmation improves correctness but may slow customer feedback. Returning `PENDING` sooner is more honest but requires user messaging and recovery. **Senior reasoning:** This F02 exercise requires **drawing and reviewing** failure behavior, not claiming that an unverified API supports idempotency or status queries.

## 6) Interview Phrasing

> "I use a sequence diagram to show the normal journey and the important exception paths. If a downstream payment call times out, I treat the result as unknown until an authoritative check resolves it; I don't show a timeout as proof of failure."

**Plain English:** The diagram explains *when* things happen and what is still uncertain. **Follow-up:** "Which state change and test would you require before allowing retries?"

# 9. L06 — Deployment view: where software runs and what can fail [C019 Core]

## 1) Purpose and prerequisites

A **deployment view** answers: “Where do program instances and data stores run, through which network boundaries, and which failure affects which parts?” Prerequisites: logical components (L03), external boundaries (L02), and sequence dependencies (L05). **[VF]** The C4 deployment guidance explains mapping instances of software systems or containers onto deployment nodes, optionally including load balancers and firewalls [S8]. A logical box is not automatically a separate deployment unit.

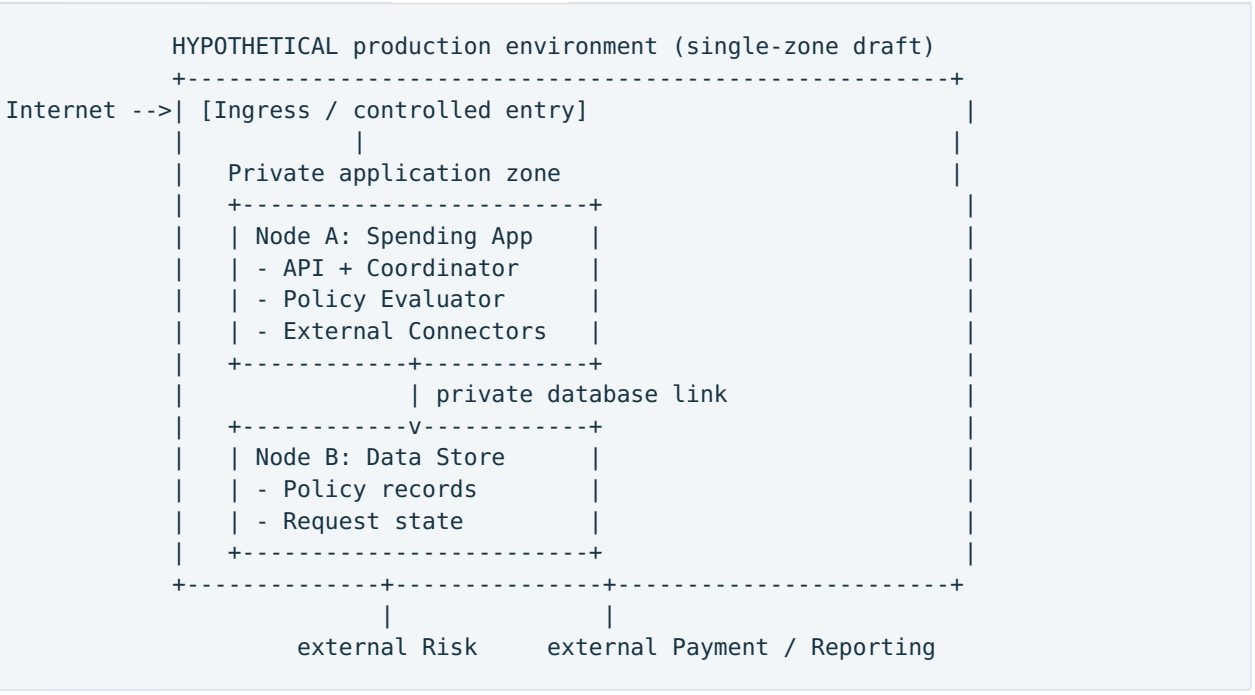
## 2) Terms and fundamentals

- **Runtime / instance:** A running copy of an application.
- **Node:** Execution location, such as a virtual machine, server, or managed runtime.
- **Environment:** A named stage such as development, test, or production.
- **Network boundary/zone:** Group of connected resources with defined access paths.
- **Database node:** Place a data store runs; it can be logically separate from the application.
- **Failure domain:** Set of resources affected together by one outage.
- **Blast radius:** The scope of business harm a fault can cause.
- **Operations owner:** Team responsible for monitoring, recovery, and support.

**Method [REC]:** Identify environment; map each logical application/data store to deployment nodes; show external paths; mark assumed trust and failure boundaries; annotate operational owners; validate the diagram with platform, security, and operations teams.

## 3) Worked example — DEP-01 [HA]

**Audience:** platform, operations, and security. **Question:** “What would a failed application runtime or database stop?”




**Read it step by step:**

1. The **same Spending Application** from L03 runs on Node A in this draft.
2. Policy records and request state map to Node B. Two logical stores may share one physical database; this **does not** merge their ownership rules.
3. External paths must be validated against network and organizational boundaries; labels are not actual configurations.
4. A single Node A is a **single point of failure** in this invented draft. A single Node B is another. That makes the diagram useful: it reveals a risk to resolve, not a production recommendation.
5. Region, replication, backup, network controls, identity mechanisms, and service support levels are **unknown**. They belong in review findings or later specialist designs, not invented facts.

## 4) Guided exercise — E06

**Goal:** Find failure boundaries. **Steps:** (a) cross out Node B; (b) explain which user journey is likely affected; (c) identify an owner and one recovery question; (d) propose a second deployment alternative as a *design option*, not as an approved architecture. **Expected:** explicit customer impact, uncertainty, and operational trade-off. **Hint:** two application instances do not remove a shared database failure.

## 5) Failure, trade-off, senior reasoning

**Failure:** A diagram shows three application replicas but hides one shared database. **Detect:** trace the data dependency of each replica. **Fix:** show database nodes and failure domains, then require operational validation. **Trade-off:** More runtime redundancy may increase availab-

ility but adds cost and consistency/operational complexity. **Senior reasoning:** Don't infer availability figures from node count. Ask for measurable requirements and failure tests from F01/T07 before finalizing a runtime architecture.

### 6) Interview Phrasing

> "The deployment view maps logical responsibilities to actual runtime and data-store boundaries. I use it to expose shared dependencies, network crossings, recovery ownership, and possible single points of failure."

**Plain English:** The deployment picture makes practical failure risk visible. **Follow-up:** "Why is a second application server not enough to remove the biggest failure risk?"

# 10. L07 — Choose views by concern; check consistency across all five [C019 Core]

### 1) Purpose and prerequisites

Different drawings should be **consistent explanations of one design**. L02–L06 taught each view separately. This lesson connects them and addresses C019's requirement to choose views **for different audiences**. **[VF]** C4 guidance distinguishes multiple diagram scopes and notes that not every real project needs every level; select diagrams by value [S2]. In **this F02 assessment**, all five named views are required by the syllabus **[SR]**.

### 2) Terms and fundamentals

- **Cross-view consistency:** The same element, responsibility, and boundary mean the same thing across drawings.
- **Abstraction level:** Amount of detail intentionally shown. Context is broad; components zoom in.
- **View inventory:** A list of each diagram, audience, decision/question, owner, and version.
- **Architecture baseline:** A dated set of diagrams, decisions, requirements, and outstanding issues reviewed together. It is **not** a guarantee of completeness.
- **Consistency invariant:** Rule that must stay true across views; for example, the *platform request-processing state* has one named authoritative owner.

**Procedure [REC]:** Name each stable element; maintain a diagram index; check six invariants: (I1) same system boundary, (I2) same named external interfaces, (I3) one proposed owner per important state, (I4) actual participant names in sequences exist in the logical inventory, (I5) deployed application/store maps to a logical element, (I6) contradictory status and timing promises are flagged rather than hidden.

## 3) Worked example — cross-view review [HA]

| Check | Source view | Compared view | Result/action |
|---|---|---|---|
| I1 System in scope | Context: Spending Platform | Component: Spending Application | **Consistent only if** Application is labeled as inside Platform; add annotation |
| I2 Payment external? | Context: external Payment | Data-flow: external Payment | Consistent with H01, not verified in a real organization |
| I3 Request state owner? | Component: Transaction State Repository | DFD: D2 Request State | Proposed consistency; **approval still needed** |
| I4 Payment response timing? | Sequence: received response before state write | DFD: F5–F6 | No conflict; DFD doesn't specify time |
| I5 Runtime exists? | Component: Spending Application | Deployment: Node A | Consistent with H08 |
| I6 Reporting handoff? | Component: dotted/undecided | DFD: F7 exists | **Open:** when is F7 durable, what happens on failure? Link ADR-01 and its test |

**Step-by-step:** Do not erase row I6. Create a tracked question. If a stakeholder asks to approve a design, a recorded unknown is more useful than an attractive but inaccurate diagram.

## 4) Guided exercise — E07

**Goal:** Detect contradictions. **Steps:** Imagine a revised deployment view puts Risk inside Node A while context says Risk is external. Explain two valid interpretations and how you would decide which is true. Then write one-page-view inventory entries for Context-01 and DEP-01 including audience, owner, and review date. **Expected:** a clearly recorded conflict, not a forced guess. **Hint:** a connection adapter may run locally while the actual Risk system remains external.

## 5) Failure, trade-off, senior reasoning

**Failure:** The context changes but the sequence and deployment diagrams stay unchanged. **Detection:** view inventory and cross-check checklist. **Fix:** change impact review tied to element names, decision IDs, and traceability rows. **Trade-off:** Automatic diagram generation may reduce drift but can obscure business meaning; hand-curated views explain intent but demand maintenance. **Senior reasoning:** Treat meaningful inconsistencies as risks to close before implementation, especially state ownership and trust boundaries.

### 6) Interview Phrasing

> “I keep a small view inventory and cross-check names, boundaries, state ownership, and failure behavior. A diagram is useful only if it answers the audience's question and agrees with the other evidence.”

**Plain English:** Five diagrams must agree about the important facts. **Follow-up:** “How would you spot documentation drift after an integration change?”

**C019 exit gate:** Without notes, explain the purpose of all five views, redraw one correct small example of each, specify the audience, show a failure path in the sequence, and defend why each view reveals a concern the others do not. **Required evidence:** E01–E07, Q01–Q07, K1–K2, and S1. Score *explain*, *apply*, and *trade-off defense* separately (all ≥4/5).

# 11. L08 — Compare realistic options with explicit criteria [C020 Core]

## 1) Purpose and prerequisites

A design choice is not justified by words like “modern,” “simple,” or “best practice.” An **option comparison** makes the goals, alternatives, constraints, uncertainty, and costs visible. Pre-requisites: F01 requirements, the relevant component/data-flow/sequence views, and the definition of a trade-off. **[VF]** Carnegie Mellon University's Software Engineering Institute (SEI) describes architecture evaluation in terms of competing quality attributes and trade-offs [S9].

## 2) Terms and fundamentals

- **Option / alternative:** A realistic way to meet a requirement.
- **Criterion:** A specific quality used to judge choices: correctness, timeliness, operability, cost, delivery effort.
- **Hard constraint / gate:** A requirement no score can override, such as valid legal access to information.
- **Weight:** Importance assigned to a criterion. Numbers should be explained, not fabricated as facts.
- **Evidence:** Test, vendor/API contract, prototype, stakeholder approval, or production measurement supporting a judgment.
- **Sensitivity:** How much the recommendation would change if a weight or assumption changed.
- **Reversibility:** How hard it is to change a decision after delivery.

**Method [REC]:** (1) write goal and constraints; (2) list two credible options, including the simplest; (3) define criteria before scoring; (4) mark evidence versus estimates; (5) check hard constraints first; (6) compare effects and ownership; (7) choose conditionally if evidence is missing; (8) record when to revisit.

## 3) Worked example — reporting integration choice [HA]

**Decision question:** How does the Spending Platform provide transaction updates to Reporting? **Assumption H05:** Finance may accept bounded delay, but its exact allowed value is unknown. **Option A:** synchronous call to Reporting during request processing. **Option B:** durable asynchronous handoff followed by later Reporting processing. Neither option automatically provides exactly-once delivery or financial correctness. Those controls require evidence and later T02/T04 work.

```
A. synchronous (request path waits)
[Request Coordinator] --> [Reporting] --> acknowledgement --> user outcome

B. asynchronous (user path does not wait for Reporting completion)
[Request Coordinator] --> [durable handoff?] ..> [Reporting]
          |                                        |
          +--> user outcome                 later status update
```

**Comparison — provisional scores, explicitly invented for teaching (1 poor, 5 strong):**

| Criterion | Weight [HA] | A score [HA] | B score [HA] | Reason and evidence gap |
|---|---|---|---|---|
| Prevent missing reporting updates | 35% | 2 | 4 | B better **only if** durable record and recovery are proven; unverified |
| Responsive employee journey | 20% | 3 | 4 | B need not wait for Reporting; performance unmeasured |
| Operational recovery clarity | 20% | 3 | 3 | Both need alerts, ownership, and replay/repair plan |
| Delivery effort | 15% | 4 | 2 | B adds implementation and monitoring work; effort unestimated |
| Direct implementation cost | 10% | 4 | 2 | Cost assumptions unverified |
| **Weighted total (out of 5)** | **100%** | **2.90** | **3.30** | *Illustration, not validated measurement* |

**Show the calculation:** A = `0.35×2 + 0.20×3 + 0.20×3 + 0.15×4 + 0.10×4 = 2.90`. B = `0.35×4 + 0.20×4 + 0.20×3 + 0.15×2 + 0.10×2 = 3.30`.

**Step-by-step interpretation:** Option B ranks higher **under invented weights and assuming durable handoff works**. It is **not yet approved**. The architectural gate is evidence that an update cannot disappear when local state is saved or when Reporting is down. A durable

queue alone does not prove atomic coordination with the local database. Ask engineering to demonstrate the mechanism and failures, and Finance to approve the freshness target. Security and authorization requirements remain non-negotiable gates, not points to trade away.

**Sensitivity probe [HA]:** If delivery time becomes a firm near-term constraint and Finance accepts the limitations of A, the ranking might reverse. Recalculate using agreed weights instead of inventing a predetermined winner.

## 4) Guided exercise — E08

**Goal:** Defend a choice without false precision. **Steps:** (a) explain what each criterion measures; (b) name the two missing pieces of evidence most likely to change Option B's ranking; (c) change the delivery-effort weight to 35% and reduce correctness weight to 15% while leaving the other weights unchanged (total still 100%); (d) recompute and interpret; (e) state one hard gate. **Expected:** transparent math, decision, and uncertainty. **Hint:** weighted scores are discussion tools, not measured probabilities.

## 5) Failure, trade-off, senior reasoning

**Failure:** Someone manipulates criteria after seeing the result. **Detect:** selection rules or weights change without stakeholder sign-off. **Fix:** publish criteria before scoring, record owners, and run sensitivity analysis. **Trade-off:** Quantitative tables improve comparability but can create fake certainty; narrative risks hidden preference. Use both. **Senior reasoning:** Ask whether design alternatives are truly viable; a fast option that violates required controls is not an option, regardless of score. Explain what new evidence would change the decision.

## 6) Interview Phrasing

> “I compare at least two workable options against agreed criteria, and separate hard constraints from preferences. I use the scores to expose assumptions, not to invent certainty, and I state what evidence would make me change the choice.”

**Plain English:** A score helps discussion; proof and business constraints still decide. **Follow-up:** “What would you do if Finance will not accept delayed reports?”

# 12. L09 — Write an architecture decision record with consequences and triggers [C020 Core]

## 1) Purpose and prerequisites

An **architecture decision record (ADR)** preserves the **reason for an important design choice** so future teams can understand, implement, challenge, or reverse it. Prerequisite: an explicit options comparison (L08). **[VF]** AWS Prescriptive Guidance describes ADRs as recording context, decision, consequences, status, and lifecycle; an accepted record may be superseded by a new record rather than silently rewritten [S10, S11]. This is AWS guidance, **not** an F02-specific mandated enterprise policy.

## 2) Terms and fundamentals

- **Context:** Business problem, relevant requirements, and constraints.
- **Decision:** What the team chose. Specific enough to implement or test.
- **Consequences:** Positive and negative results, including operational and delivery obligations.
- **Status:** Lifecycle stage, such as Proposed, Accepted, Rejected, or Superseded (our suggested vocabulary **[REC]**).
- **Revisit trigger:** Observable event or changed assumption that requires reconsideration.
- **Owner / approver:** Person/team preparing and maintaining the ADR versus party with authority to approve it. They may differ.
- **Supersede:** Replace an accepted decision with a new, linked decision while keeping the history.
- **Compliance/evidence check:** Proof implementation meets the decision, not just prose saying it does.

**How to write [REC]:** Give a stable ID, title and date; define the scope; cite requirements; list alternatives and criteria; state the selected design in active voice; record consequences, validation plan, owners and triggers; link diagrams and dependencies; obtain the needed review. Keep it short enough to read.

## 3) Worked example — full ADR-01 [HA + REC]

> **ADR-01 — Reporting handoff for purchase-request updates**
> **Status:** Proposed, **not approved**.
> **Date:** Illustrative only: 2026-10-24.
> **Author:** Fictional Solution Architecture Team.
> **Decision approvers:** Finance product owner and architecture decision owner (**actual people not identified**).
> **Related requirements:** R03 draft reporting freshness; R02 persist platform request-processing state.
> **Related views:** Component-01, DFD-01 F7, SEQ-01, DEP-01.
> **Context:** Finance wants visibility of purchase requests while employees should not depend on Reporting to submit them. The acceptable delay and durability mechanism are not yet verified (H05–H07).
> **Alternatives:** A. synchronous request-path reporting call; B. durable asynchronous handoff; compare L08.
> **Decision (conditional proposal):** Use **B** *if* engineering demonstrates durable handoff and recovery, security approves relevant data fields, and Finance approves a freshness target. Until these conditions are met, keep status Proposed and do not present B as an accepted design.
> **Positive consequences:** Reporting can be processed without blocking the employee request path; outages may be buffered if durability and capacity are proven.
> **Negative consequences:** Extra component and operational monitoring; delayed Finance visibility; backlog/replay and duplicate-handling questions; unclear atomicity until designed and tested.
> **Validation:** Inject a Reporting outage, observe whether acknowledged request-state changes are eventually represented exactly as contractually defined; test restart, duplicate notification, lag measurement, and access controls. **The required guarantees are not yet specified**.
> **Risks:** RK03 missing update; RK04 stale report; RK05 unauthorized fields.
> **Dependencies:** DP03 durable handoff design/contract, DP04 reporting intake agreement, DP05 Finance freshness approval.
> **Revisit triggers:** Finance approves a shorter delay than the chosen approach can meet; the agreed failure test loses updates; recurring backlog exceeds agreed freshness; reporting volume or support ownership changes materially.
> **Change history:** v0.1 proposed for review. If accepted then materially changed, link a new superseding ADR rather than silently erasing why the old choice was made.

**Read it step by step:** (1) Title says what is decided; (2) status avoids an unearned approval claim; (3) context explains why; (4) options show rejected alternative; (5) decision has conditions; (6) consequences include costs; (7) tests and owners turn words into action; (8) triggers make revision possible.

**Compact blank ADR template for reuse:**

```
ADR-[ID]: [Decision question]
Status: Proposed / Accepted / Rejected / Superseded
Date/version: [...]       Author/owner: [...]       Approver: [...]
Business goal and requirement IDs: [...]
Context, constraints, assumptions, open facts: [...]
Options and explicit evaluation criteria: [...]
Decision and scope: [...]
Positive consequences: [...]
Negative consequences and residual risk: [...]
Validation tests/evidence: [...]
Dependencies and accountable owners: [...]
Revisit triggers and review date: [...]
Related views / supersedes: [...]
```

## 4) Guided exercise — E09

**Goal:** Write a second decision record. **Steps:** Draft ADR-02 comparing (A) a single owner for platform request state with (B) multiple independent writers; include two benefits/risks, approval and validation plan, and at least two measurable revisit triggers. **Expected:** decision record, not implementation code. **Hint:** one owner for the *platform request state* does not make that owner authoritative for all payment lifecycle facts.

## 5) Failure, trade-off, senior reasoning

**Failure:** “We will use messaging because it scales.” No business goal, constraints, alternative, responsibility, or trigger. **Detect:** the ADR cannot answer “why this decision?” or “what if it fails?” **Fix:** state measurable concern, conditions, negative consequences, owner, and evidence. **Trade-off:** Too many ADRs become unreadable; too few lose significant decisions. **Senior reasoning:** Use ADRs for consequential, hard-to-reverse decisions, not every coding convention. Avoid rewriting history to make an old choice appear obvious.

## 6) Interview Phrasing

> “For a consequential choice, I write a short ADR with the problem, alternatives, selected option, expected consequences, accountable owner, and a condition that would trigger reconsideration. The record explains why, not just what.”

**Plain English:** A future architect should understand the trade-off without having attended the meeting. **Follow-up:** “What is the process when new evidence makes an accepted ADR wrong?”

**C020 exit gate:** Explain and demonstrate **both** option evaluation and a complete ADR. Produce E08–E09, Q08–Q09, K3, and S2. Independently defend alternate assumptions, consequences, and revisit triggers. Score explain/apply/trade-off (all ≥4/5).

# 13. L10 — Trace requirements to decisions, tests, risks, dependencies, and owners [C021 Core]

## 1) Purpose and prerequisites

**Traceability** answers: “Why does this design element exist, how will we test it, what can defeat it, what does it depend on, and who is accountable?” It is the link from F01 requirements to implementation evidence. Prerequisites: requirements, diagrams, and ADRs (L02–L09). **[VF]** NASA's Systems Engineering Handbook describes a requirement verification matrix with unique requirement identifiers, sources, and defined verification approaches [S12]. NASA software architecture guidance also discusses requirements-to-architecture traceability [S13]. The extended columns here are **our F02 teaching format**, not a claimed NASA-prescribed table schema.

## 2) Terms and fundamentals

- **Requirement ID (Rxx):** Stable identifier independent of document page order.
- **Design decision ID (ADR-xx):** Link to the rationale for an architectural choice.
- **Test ID (Txx):** Repeatable check with expected result and evidence.
- **Risk ID (RKxx):** Named threat to satisfying the requirement.
- **Dependency ID (DPxx):** Internal or external prerequisite.
- **Accountable owner:** One named role or team that will close a gap; “everyone” is not accountable.
- **Forward trace:** Requirement → design → verification.
- **Backward trace:** Design or test → requirement/concern that justifies it.
- **Coverage gap:** Missing decision, test, risk handling, dependency or owner.
- **Orphan artifact:** Test/design with no valid requirement or recorded rationale.

**Method [REC]:** (1) uniquely label each requirement; (2) point to the relevant view and decisions; (3) specify at least one verifiable test; (4) state risk, dependencies, owner, current status; (5) review from requirement to evidence and from artifact back to requirement; (6) update whenever a requirement changes.

## 3) Worked example — TRACE-01 [HA]

| Req ID and requirement | Relevant views / ADR | Test ID and expected proof | Risk and detection | Dependency | Accountable owner | Current evidence |
|---|---|---|---|---|---|---|
| **R01** Reject requests that violate stored spending rules | Component-01 / SEQ-01; **ADR-02 proposed** | **T01:** over-limit request gets no Payment submit | **RK01:** policy bypass; observe calls and audit | **DP01:** policy owner confirms rule and test input | Policy product owner | **Not run** |
| **R02** Persist the platform's request-processing outcome with a stable request ID | Component-01 / DFD-01 / SEQ-01,02; **ADR-02 proposed** | **T02:** response + persistence tests including storage fault; no falsely confirmed result | **RK02:** accepted action not recorded; correlate IDs and investigate | **DP02:** transaction-state authority sign-off | Transaction domain owner | **Not run** |
| **R03** Make request status visible to Finance within an **unapproved 5-minute example target** [H07] | DFD-01 F7 / DEP-01 / **ADR-01 proposed** | **T03:** after Reporting outage, verify data recovery and measured freshness vs. approved target | **RK03:** missing/stale report; freshness gap alert | **DP03–05:** durable handoff, interface, Finance approval | Finance product owner | **Blocked by missing target/contract** |
| **R04** Share only authorized request data across external interfaces | DFD-01 / DEP-01; security design **not yet approved** | **T04:** prohibited fields/access are rejected; reviewer checks actual contract | **RK05:** unauthorized disclosure; review access logs and payload fields | **DP06:** security/data classification approval | Security control owner | **Not run** |

**Walkthrough:**

1. R01 is supported by component and sequence views; its test catches a call that should not happen.
2. R02 links owner and error state; “store the status” is not enough unless failure paths are checked.
3. R03 is **not ready**: the number is invented, and the reliability mechanism is unproven. A trace row may document an intentional blocker.
4. R04 is a constraint requiring specialist review; the table does **not** certify security.
5. When examining **ADR-01 backward**, you can ask which requirement motivates it (R03), which tests demonstrate it (T03), and which risks might invalidate it (RK03).

**Traceability check procedure [REC]:** Each mandatory requirement should have a clear design reference, test, risk or explicit risk-review reason, dependency or explicit “none,” an accountable owner, and evidence status. Include tests' expected results, not only test names. A filled-in row with invented links is worse than an honestly marked gap.

### Optional executable Python — find incomplete trace rows

Python is included only as an F02 evidence-checking aid; it is **not** a replacement for the B01/T11 programming curriculum. Requires **Python 3**, using only built-in language features. Copy this block into `trace_check.py`; run `python3 trace_check.py` (or `python trace_check.py` if that is your Python 3 command).

```
required_fields = ("decision", "test", "risk", "dependency", "owner")

rows = {
    "R01": {"decision": "ADR-02", "test": "T01", "risk": "RK01",
            "dependency": "DP01", "owner": "Policy Owner"},
    "R02": {"decision": "ADR-02", "test": "T02", "risk": "RK02",
            "dependency": "DP02", "owner": "Transaction Owner"},
    "R03": {"decision": "ADR-01", "test": "", "risk": "RK03",
            "dependency": "DP05", "owner": "Finance Owner"},
}


def missing_links(requirements):
    gaps = {}
    for req_id, links in requirements.items():
        absent = [field for field in required_fields if not links.get(field)]
        if absent:
            gaps[req_id] = absent
    return gaps


assert missing_links({"X": {name: "ok" for name in required_fields}}) == {}
assert missing_links(rows) == {"R03": ["test"]}
print(missing_links(rows))
```

**Line-by-line explanation (grouping literal data rows):**

1. `required_fields = (...)` defines the five trace links this simple checker requires.

2. `rows = { ... }` creates a Python **dictionary**: a collection mapping unique keys to values. `R01`, `R02`, and `R03` are requirement IDs.
3. Each inner `{...}` maps a link type to its reference; an empty string `""` means a missing test reference for R03.
4. `def missing_links(requirements):` defines a reusable **function** accepting a dictionary.
5. `gaps = {}` starts an empty dictionary for findings.
6. `for req_id, links in requirements.items():` loops through each requirement and its links.
7. `absent = [...]` is a **list comprehension**: it builds a list of required field names whose values are missing or empty. `links.get(field)` looks up the field without crashing if it is absent.
8. `if absent:` checks whether that list contains any gap.
9. `gaps[req_id] = absent` records that requirement's missing fields.
10. `return gaps` gives the completed findings back to the caller.
11. The first `assert` checks a complete record yields no gaps. `assert` stops execution if a check fails.
12. The second `assert` checks that the intentionally incomplete R03 has a missing `test` link.
13. `print(...)` shows the finding, expected output: `{'R03': ['test']}`.

**Limitations:** This simple script does **not** verify whether an ADR/test ID exists in another repository, a test passed, an owner accepted responsibility, a link is accurate, or security is adequate. It does not count explicit “none” policy as sufficient proof. These require human review and/or a more complete data model. **[VF]** Python's official language documentation supports the built-in collections and assertions used here [S14].

## 4) Guided exercise — E10

**Goal:** Demonstrate forward and backward traceability. **Steps:** (a) run the code and explain the output; (b) add R04 with an empty owner and verify the checker detects it; (c) fix R03 using `T03` only after adding an evidence plan; (d) find a hypothetical orphan test `T99` and explain why this checker alone will not find it; (e) identify the biggest unresolved blocker to R03. **Expected:** a trace row, a detected gap, and a reasoned human check. **Hint:** a test *link* is not a *passed test*.

## 5) Failure, trade-off, senior reasoning

**Failure:** A high-priority requirement is omitted from diagrams but test plans still look complete. **Detect:** start from all requirement IDs and audit forward, then reverse from decisions and tests. **Fix:** add a design response, test, reviewer, and owner or reopen the requirement. **Trade-off:** Detailed matrices cost maintenance time; overly sparse matrices hide gaps. **Senior reasoning:** Focus on material requirements and risks while preserving **every** mandatory F02 linkage. The architecture is not “done” because each row contains text; approval requires trustworthy evidence.

### 6) Interview Phrasing

> "I trace each important requirement to its architecture decision, verification test, risk, dependency, and accountable owner. I check in both directions so we can find missing coverage and decisions that no longer serve a business need."

**Plain English:** Every design choice needs a reason, test, risk view, and owner. **Follow-up:** "How would you handle a critical requirement that has a test ID but no passing test evidence?"

**C021 exit gate:** Complete E10/Q10/K4/S3; show both forward and backward traces; explain a missing or wrong link and fix it; defend the level of detail. Score explain/apply/trade-off (all ≥4/5).

# 14. L11 — Formal architecture review: people, agenda, evidence [C022 Depth]

## 1) Purpose and prerequisites

An **architecture review** asks whether the design addresses important business and quality requirements, where it is risky, and what evidence or changes are still needed. Prerequisites: five views, options and ADRs, and a traceability matrix (C019–C021). **[VF]** SEI's Architecture Tradeoff Analysis Method (ATAM) uses business drivers, quality scenarios, architecture approaches, and analysis of risks/trade-offs [S9]. AWS recommends lightweight, blame-free reviews that end in prioritized actions [S15]. **Our simulated review below borrows these principles; it is not certified ATAM execution or a claim about company-specific governance.**

## 2) Terms and fundamentals

- **Formal (for this lesson):** Scheduled review with an agenda, named reviewers, pre-read, criteria, recorded findings and a decision. It need not be bureaucratic.
- **Design review package / pre-read:** Diagrams, decisions, requirements, risks, and open questions sent ahead.
- **Reviewer:** Person who evaluates the design from a particular concern (e.g., finance data, security, operations).
- **Decision authority:** Role authorized to approve, defer, or require action. A facilitator is not automatically this authority.
- **Finding:** Evidence-backed observation with severity, owner, requested resolution, and due date.
- **Quality scenario:** Specific situation plus expected system response and measurable target.

- **Risk acceptance:** Explicit authorized choice to proceed despite a known remaining risk; not a substitute for mandatory controls.

**Review method [REC]:**

1. Define scope and decision requested: “Ready to implement the reporting handoff?”
2. Invite product/business owner, software architect, engineering, finance/data, security, operations, QA, and external interface owner as appropriate. One role may cover more than one concern if authority is clear.
3. Send a concise pre-read: context, component, DFD, sequence normal/failure, deployment, ADRs, traceability, risks, questions.
4. Ask reviewers to challenge assumptions and identify *missing evidence*, not to vote on visual beauty.
5. Review high-business-impact failure and data questions first.
6. Log finding, severity, responsible owner, evidence, proposed action, retest and decision date.
7. Record whether implementation can start, must be limited, or is blocked; document the exact scope and authority.

## 3) Worked example — REVIEW-01 [HA + REC]

**Scope:** Review only the proposed purchase-request processing and Reporting integration; **not** the full payment domain or production system.

**Suggested 45-minute agenda (teaching example, not official timing):**

| Minutes [HA] | Question | Speaker / reviewer |
|---:|---|---|
| 0–5 | Which business goal and requirements? | Product/architect |
| 5–13 | Boundaries, state owner, data flow? | Architect/data/finance |
| 13–23 | Normal path, timeout, storage failure? | Engineering/QA/payment interface owner |
| 23–30 | Data access and runtime failures? | Security/operations |
| 30–38 | ADR alternatives and evidence gaps? | Decision owner |
| 38–45 | Findings, accountable actions, readiness? | Chair/authorized approver |

**Review questions for each view:** Context: Which systems are external? Component: Who owns processing state? DFD: What crosses trust boundaries? Sequence: What happens after timeout or failed state write? Deployment: What fails if the shared database node is unavailable? ADR: Which option and under which conditions? Traceability: Which tests are missing and who closes the gaps?

## Fictional finding log:

| Finding | Severity [REC] | Evidence | Required action | Owner (role) | Status |
|---|---|---|---|---|---|
| F-01 No approved owner of platform request state | **Blocker** | H04 + Component-01 proposal | Obtain explicit owner/authority and update C019/C021 artifacts | Transaction domain lead | Open |
| F-02 Reporting durability not proven | **Blocker for B implementation** | ADR-01 condition + SEQ-02 ambiguity | Define durable handoff and test failure paths | Integration engineering lead | Open |
| F-03 Finance freshness target unapproved | **Major** | R03/H07 | Obtain target and Finance acceptance evidence | Finance product owner | Open |
| F-04 Node B single point of failure | **Major / scope-dependent** | DEP-01 | Evaluate against agreed reliability need; prove recovery or record accepted scope risk | Operations/platform owner | Open |
| F-05 Request payload field access not reviewed | **Blocker for sensitive-data flow** | DFD-01 F1/F7 | Complete security/data-contract review | Security/data owner | Open |

**Notes:** A finding severity is a recommendation specific to this invented example, **not** an enterprise policy. A "blocker" for one implementation can coexist with permission to do lower-risk discovery/prototyping. Security and finance authority must not be silently delegated to an architect.

### 4) Guided exercise — E11

**Goal:** Facilitate a structured review without pretending all facts are known. **Steps:** (a) choose five reviewers/roles and give each one question; (b) write one additional evidence-based finding about retry uncertainty; (c) classify it and name an accountable owner; (d) distinguish a needed experiment from a diagram correction. **Expected:** a review record with owners and follow-up tests. **Hint:** "Looks fine" is not evidence.

### 5) Failure, trade-off, senior reasoning

**Failure:** Review meeting becomes a style argument and critical data ownership remains unresolved. **Detect:** no link from discussion to requirements, risks or tests. **Fix:** focus on high-impact scenarios; time-box minor notation disputes; write actionable findings. **Trade-off:** More reviewers catch more risks but cost time and may delay delivery. **Senior reasoning:** Review as early as needed for irreversible choices, and revisit when material constraints change; keep decision rights explicit.

### 6) Interview Phrasing

> "I run architecture reviews against business drivers and quality scenarios, not diagram aesthetics. I bring the right decision owners, document material findings, and finish with accountable actions and clear readiness criteria."

**Plain English:** The goal is to find risky gaps before teams build them. **Follow-up:** "How do you handle a review where engineering wants to proceed but the data owner has not agreed to state ownership?"

# 15. L12 — Decide whether the design is ready to implement [C022 Depth]

### 1) Purpose and prerequisites

A design can be *visually clear* but **not yet implementable**. A readiness decision turns review findings into an accountable go/no-go/scope-limited judgment. Prerequisites: review process and findings (L11), tests and owners (L10). **[VF]** AWS emphasizes reviewing early, identifying

significant issues, and leaving with improvement actions; SEI evaluation highlights quality trade-offs and risks [S15, S9]. The gate categories below are **a recommended F02 exercise structure**, not a universal standard.

## 2) Terms and fundamentals

- **Ready:** Scope, critical decisions, ownership, and evidence are sufficient to begin the *specified* implementation work.
- **Conditionally ready:** A limited, reversible part may start while clear conditions protect higher-risk work.
- **Deferred / not ready:** A blocking uncertainty or missing control prevents responsible implementation of the requested scope.
- **Exit criterion:** Observable proof required to close a finding.
- **Waiver / exception:** Time-bound authorization to deviate from an approved rule, given by the appropriate decision owner. Whether it is allowed is organization-specific and unverified here.
- **Residual risk:** Risk remaining after planned actions; it still requires ownership.

**Readiness checklist [REC]:**

| Gate | Passing evidence expected | If missing |
|---|---|---|
| Business scope | Named goal, priority requirements, approved acceptance meaning | Return to F01/product owner |
| Boundaries/ownership | Views agree; authoritative processing-state owner recorded | **Block state-changing implementation** |
| Correctness/failure | Normal and exception sequences; evidence plan for timeout and write failures | Block risky external side effects |
| Data/access | Data minimization, permissions, owners, classification review | Block unapproved sensitive-data flow |
| Decision quality | ADR with alternatives, consequences, approver, revisit trigger | Reopen decision |
| Traceability | Requirements linked to design, tests, risks, dependencies and owners | Assign gaps and close critical rows |
| Operations | Deployment/failure boundaries and support owners known at required depth | Limit scope or defer runtime release |
| Findings | Blocking items closed with evidence; accepted residual risks signed by authorized parties | Defer protected scope |

**Possible decisions [REC]:** `READY FOR SPECIFIED IMPLEMENTATION`, `CONDITIONALLY READY FOR LIMITED WORK`, or `NOT READY / DEFER`. Never issue an unconditional "approved" statement on the evidence in this sample.

## 3) Worked example — REVIEW-01 outcome [HA]

**Hypothetical decision: NOT READY** for building the full state-changing purchase-to-reporting integration, because F-01, F-02 and F-05 are unresolved. **CONDITIONALLY READY** for discovery tasks only: confirm owners and contracts, create a test harness using synthetic data, and draft updated diagrams/ADRs. No external financial action, production data flow, or deployment approval is included.

**Required actions and success measures [HA]:**

| Action | Accountable owner (fictional role) | Verification evidence | Review trigger |
|---|---|---|---|
| Confirm request-state authority | Transaction domain lead | Approved data ownership entry and updated diagrams | Ownership sign-off received |
| Demonstrate no acknowledged update is silently lost under tested failures | Integration engineering lead | Approved test plan + observed results on outage/restart | Test results recorded |
| Confirm data fields and access rules | Security/data owner | Approved data-flow contract and negative-access tests | Review completed |
| Confirm reporting freshness | Finance product owner | Agreed measure and threshold, replacing H07 | Signed requirement baseline |
| Confirm recovery scope for Node B | Platform/operations owner | Failure/recovery test or formally approved design change | Evidence reviewed |

**What can be measured later [REC]:** percentage of critical requirements with passing evidence; count of open blockers; accepted-versus-unowned risks; freshness against the *approved* target; recovery test result. No measurements have been fabricated.

## 4) Guided exercise — E12

**Goal:** Produce a defensible readiness memo. **Steps:** (a) choose ready/conditional/not ready for the full integration; (b) list 3 blocking facts, 2 allowed discovery tasks, and 2 forbidden commitments; (c) name who can approve changed scope; (d) write 3 observable exit criteria. **Expected:** precise decision with limits and a re-review plan. **Hint:** a beautiful diagram is not test evidence.

## 5) Failure, trade-off, senior reasoning

**Failure:** Reviewers mark an unresolved financial or data-control risk "accepted" without its actual owner. **Detect:** vague ownership and no documented acceptance authority. **Fix:** hold the protected scope; obtain real accountable approval and close hard gates. **Trade-off:** Early

implementation can accelerate learning, but premature irreversible behavior makes rework and harm harder to control. **Senior reasoning:** Approve *discovery or reversible work* without disguising it as approval for sensitive, state-changing production features.

## 6) Interview Phrasing

> “A design is ready only for the scope its evidence supports. I separate implementation blockers from lower-risk discovery work, assign owners to each gap, and require explicit exit criteria before the full design proceeds.”

**Plain English:** You can continue learning without pretending the risky system is approved. **Follow-up:** “Would you let a team start coding while an external payment-state owner is still disputed?”

**C022 Depth exit gate:** Complete E11–E12, Q11–Q12, K5, and S4 as a **separately recorded** depth result. The full C022 review may be deferred until scenario or interview evidence makes this depth relevant. It must not disappear from the audit.

# 16. Assessment exercises — do these before opening the answer key

## A. One cumulative F02 portfolio task (K1–K5) [SR]

**Case:** Use the H01–H09 fictional Spending Platform. The business says: “Employees should see the status of a submitted purchase request; Finance needs timely visibility; risks and ownership must be clear.” Do not assume any actual vendor contract or company-specific architecture.

**Submission format [REC]:** One reference packet of diagrams, decision records, and tables. For interview rehearsal, give a **ten-minute verbal walkthrough** (source-required practice). The checklist below describes evidence, not auto-completed work.

- **K1 — C019:** Draw **all five** views from scratch: context, component, data-flow, sequence, deployment. Each has title, one audience, one question, boundaries, H-assumptions, and an owner/question register. Draw the **normal transaction request**.
- **K2 — C019:** Draw one separate **failure sequence** for a timeout after the external Payment system may have acted. Cross-check state owner, data destination, and runtime mapping across views. Give the mandated **10-minute verbal walkthrough**: scope (1 min), context/components (2 min), data flow/ownership (2 min), happy and failure paths (2 min), deployment (1 min), trade-offs/open issues (2 min). Timing is a suggested allocation **[REC]**, not an official interview rule.
- **K3 — C020:** Compare **at least two plausible** reporting approaches with explicit criteria, hard gates, evidence gaps, and costs. Write a complete proposed ADR with decision, positive/negative consequences, named approver, and **observable revisit triggers**.
- **K4 — C021:** Trace at least R01–R04 to views/ADRs, tests with expected results, risks, dependencies, and accountable owners. Perform a reverse check from ADR-01 and T03. Mark evidence not yet available.
- **K5 — C022 Depth:** Simulate one formal architecture review using reviewers, questions, finding log, statuses, and a readiness disposition (ready/conditional/defer). Explicitly separate work that may begin from work that must not begin.

**Portfolio acceptance:** There is *no* automatic pass for this template. The learner must submit a real self-generated response and be scored. Use §18 and §17.

## B. Knowledge and application questions (Q01–Q12)

Answer without notes, with **one example** and **one trade-off or failure** in each answer.

| ID | Question | Coverage |
|---|---|---|
| Q01 | Distinguish stakeholder concern, viewpoint, and view. Why not use one mega-diagram? | C019/L01 |

| ID | Question | Coverage |
|---|---|---|
| Q02 | What belongs in a context view? What must it deliberately leave out? | C019/L02 |
| Q03 | Does a component box mean a microservice? Who owns the platform's request-processing state? | C019/L03 |
| Q04 | How does a data-flow view differ from a sequence view? Name one trust-boundary question. | C019/L04 |
| Q05 | A Payment request times out. What do you know, what do you not know, and what status should be shown? | C019/L05 |
| Q06 | What does deployment reveal that a component diagram cannot? Explain a shared database failure. | C019/L06 |
| Q07 | Two views disagree on where Risk runs. How do you resolve rather than guess? | C019/L07 |
| Q08 | Why is a weighted criteria score not proof? Which hard gates override scores? | C020/L08 |
| Q09 | List an ADR's mandatory teaching fields; explain an observable revisit trigger and superseding a decision. | C020/L09 |
| Q10 | Trace R03 forward and ADR-01 backward. What's the difference between test linkage and test evidence? | C021/L10 |
| Q11 | What must a formal review produce besides a discussion? How do reviewers challenge a state owner? | C022/L11 |
| Q12 | What does *conditionally ready* mean? Which risks cannot be accepted by the wrong approver? | C022/L12 |

## C. Senior decision challenge — S1–S4 [SR]

### Interview prompt from the authoritative F02 card:

> “A proposed design looks simple but omits an owner for transaction state. Choose the smallest additional architecture views and tests needed for a design-review decision.”

**Scenario adjustment [HA]:** The candidate already has a clean Context-01 and a *happy-path* sequence. There is an external Payment dependency, a proposed Reporting connection, and no proof of who may update the platform request-processing state. Engineering wants to start implementation tomorrow.

**Required response; address all seven dimensions:**

1. **Business goal:** State who needs which reliable outcome.
2. **Assumptions / missing facts:** Name the state authority and provider-contract gaps without inventing facts.
3. **At least two realistic options:** For example, one accountable state writer with read-only projections vs. multiple services independently writing the same state, or defer integration vs. begin low-risk discovery.
4. **Decision and trade-off:** Choose the smallest **additional** views/annotations and tests that could resolve the current uncertainty. Explain why you did *not* produce an indiscriminate new design set.
5. **Failure, security, and data implications:** Analyze timeout, unsuccessful state write, in-appropriate state access, and Reporting inconsistency.
6. **Evidence / experiment:** Define tests, owner sign-off, and a measurable outcome before saying ready.
7. **Execution:** Name approvers, action owners, review date/event, and a recorded implementation-readiness decision.

**Separate scored subtests:**

- **S1 — C019:** Select a minimal additional logical state-ownership/component annotation and an exception sequence, adding a focused data-flow/trust-boundary excerpt only if needed. Explain which existing views remain sufficient and why deployment evidence may still be needed for go-live (not necessarily for a narrow ownership decision).
- **S2 — C020:** Compare two credible ownership/reporting choices with criteria and a proposed ADR including revisit triggers.
- **S3 — C021:** Produce a compact state-ownership requirement-to-decision/test/risk/ dependency/owner trace row and reverse-check it.
- **S4 — C022 Depth:** Run a reviewer decision on whether *the proposed implementation scope* can start, and identify who must close blockers.

**Changed-constraint probe:** After the first recommendation, assume Finance now requires evidence-backed status freshness tighter than H07 and Payment cannot confirm any supported status-query operation. Re-state what changes, what remains unverified, and which decision must be revisited. These are **invented constraints**, not real interfaces or product promises.

**Senior answer length target [REC]:** 3–5 minutes for the core decision plus follow-up, with enough specificity to defend the choice. Written work may be longer.

# 17. Separate answer key — sample solutions, not learner results

**Use only after attempting §16 and the lessons' guided exercises.** These are *example responses to a hypothetical case [HA + REC]*. They are not facts about a real company, an actual payment interface, or tested production behavior. Equivalent answers can earn full marks if they explain, apply, and defend the choice.

## A. Prerequisite check (PR-1–PR-4)

- **PR-1:** Administrator sets policy; employee requests a purchase; Finance reviews transaction reports. Each is a *user role*, not necessarily a separate application.
- **PR-2:** "When a purchase amount exceeds the effective policy limit, the platform must return a decline and must not submit that request to Payment." Actual rule priorities and exception handling remain open.
- **PR-3:** "When Finance opens the status report during normal operations, status changes appear within an **agreed** freshness target, measured from a defined event timestamp to visibility in the report." Do not invent an approved threshold.
- **PR-4:** "Reporting may be delayed" is a provisional assumption; Finance product owner must confirm the maximum acceptable delay.

If these concepts are unfamiliar, complete F01 first. A short bridge does not replace F01 assessment.

## B. Guided exercise solutions (E01–E12)

**E01 — Viewpoint and audience.** Concern: "Which party owns the request's processing state?" Audience: data/domain lead, architect, implementer. First artifact: component view annotated with the writer/reader responsibility. Follow with data-flow for stores and movement; deployment answers *where* the store runs, not *who* owns the business fact. **Trade-off:** a single owner clarifies accountability but may require coordination when multiple clients need updates.

**E02 — Context and boundary.** The new context still has **one** central Spending Platform box. Add `[Auditor] -> [Spending Platform]: request permitted audit view` and `[Spending Platform] -> [Finance Export System]: export approved request status`. Mark both the human Auditor and the Finance Export System **outside** the platform boundary in this hypothetical. The auditor is a *role*; Finance Export is a *system*. The `Policy Validator` source-code class does **not** belong in a context view because it is internal implementation detail. Confirm whether Finance Export actually exists and what data it may receive; the invented flow does not imply authorization.

```
        [Auditor]                [Finance Export System]
            |                              ^
 request audit view              export status subset
            v                              |
+-------------------------------------------+
|       SPENDING PLATFORM (in scope)        |
+-------------------------------------------+
       [other actors / systems omitted]
```

**Failure/trade-off:** Hiding the external Finance Export boundary can hide its security owner; making the context too detailed makes it unreadable.

**E03 — Audit Writer, state ownership, and logical/physical split.** Add `[Policy Administration] -> [Audit Writer] -> [Policy-change Audit Store]` inside the draft application as **logical components**, *not* as an automatically separate deployable service. A minimum audit entry contains a change identifier, the actor's identifier, the affected policy ID, the old/new value or approved change description (subject to data-minimization review), and the recorded timestamp. Audit events are historical evidence; they are **not** the current policy value. Show the current Policy Store separately. Add the annotation: `Platform request-processing state: Transaction domain owner proposed; approval pending (H04)`. Choose a separate service only after evaluating independent ownership/change, reliability and operational cost in T01. **Failure/trade-off:** missing audit writes can hide policy changes; more deployable components may isolate change but increase operating effort. The F02 task is to name the responsibility and owner, not fully design atomic audit guarantees.

**E04 — Boundary crossings and minimization.** Mark *every* outward/inward crossing shown in DFD-01, not just Reporting (see diagram F-labels). For example, request metadata moves to external Risk and Payment, and an **approved subset** moves to external Reporting; the receipt/identity inputs crossing into the platform also need labeling. Treat F7 to Reporting as **potentially sensitive** until classified by the data/security owner. For the exercise, propose exactly three Finance-facing fields: `request_id`, `processing_status`, and `status_updated_at`. Do **not** assume they are sufficient, authorized, or permitted by the real interfaces. **Data owner:** proposed Transaction domain owner for the processing-status fact; Finance owner for report meaning. **Unverified security question:** is the recipient allowed to receive request-level status for this particular organization/tenant? **Detection:** review an actual contract and negative-access tests; an arrow alone is not a control. **Trade-off:** fewer fields reduce exposure but may prevent Finance from matching a record without another approved reference.

**E05 — Policy-decline branch and failed local write.** The first branch belongs **before** the external Payment call:

```
Employee -> Coordinator -> Policy Evaluator
                         <- policy violation
Employee <- DECLINED BY POLICY
Payment: NOT CALLED
```

For the second branch, Payment returns a stated outcome, but **State Store fails before recording it**:

```
Coordinator -> Payment: submit request REQ-17
Coordinator <- Payment: response recorded in transit (exact semantics unverified)
Coordinator -> State Store: save linked processing state
Coordinator <- State Store: WRITE FAILED
Employee    <- Coordinator: cannot confirm durable local status; investigate
```

The system may have *seen* a provider response, but it does **not** have evidence of a durable local state record. Do not tell the user that Payment definitely failed. Track the unresolved request ID and follow the confirmed operational/partner contract before retry or reconciliation. **Failure/trade-off:** returning a simple `FAILED` status may feel responsive but incorrectly imply no external action. Detailed duplication protections are T02, not assumed here.

**E06 — Node B unavailable.** Cross out **Node B (Data Store)** in DEP-01. The coordinator on Node A may still run, but it cannot reliably read current policy records or persist new request-processing state; therefore the **purchase-submission journey** must not be claimed healthy. The precise fail-safe behavior requires an approved business/technical design. **Owner:** proposed platform/operations owner investigates restoration; transaction domain owner decides what state results are trustworthy. **Recovery question:** How will records or uncertain requests be restored/verified after Node B returns? **Option A:** keep the single database node and accept a constrained, tested recovery objective if business allows it. **Option B:** design a properly controlled redundant data-store deployment, with owner, cost, consistency and tested failover. Neither is already approved. **Trade-off:** extra redundancy can improve resilience but adds complexity/cost and still requires failure evidence.

**E07 — Two plausible interpretations and inventory.** In the changed deployment drawing, `[Risk]` inside Node A could mean **(1)** the *local Risk connector/adapter* was mislabeled as the external Risk service; then the context is still conceptually consistent after renaming it `Risk Adapter`. Or **(2)** the design boundary has actually changed to host Risk evaluation inside the platform; then context, responsibilities and security boundaries must be reviewed together. Ask the Risk system owner and the platform/deployment owner to confirm; log an unresolved boundary decision until evidence arrives. Suggested inventory entries:

| View | Audience | Question | Proposed documentation owner | Review event/date |
|---|---|---|---|---|
| Context-01 | Business/product and integration | Which users and external systems are in scope? | Solution architect | On boundary confirmation; target review date **not yet agreed** |
| DEP-01 | Operations/platform/security | Which node hosts which runtime and can fail? | Platform architect | On runtime validation; target review date **not yet agreed** |

**Failure/trade-off:** a local adapter can look like an external Risk system if labels are ambiguous. Maintaining an inventory costs a little time but stops important boundary drift. Do not invent a signed review date.

**E08 — Option scores and hard gates.** Each score describes a provisional preference, **not** measured reliability or likelihood. Most important gaps for B: proof that a saved state change results in a recoverable reporting update, and an agreed Finance freshness target/operating contract. The changed weights are 15% correctness, 20% employee responsiveness, 20% recovery clarity, 35% delivery effort, 10% cost. New totals:

```
A = 0.15*2 + 0.20*3 + 0.20*3 + 0.35*4 + 0.10*4 = 3.30
B = 0.15*4 + 0.20*4 + 0.20*3 + 0.35*2 + 0.10*2 = 2.90
```

The preference *reverses*, proving the decision is sensitive to weights. But no option may bypass mandatory data access controls or falsely acknowledge a transaction. Check hard gates before comparing totals. Finance should confirm goals; engineering should test mechanisms; accountable decision owners approve the ADR.

**E09 — Sample ADR-02.**

> **ADR-02: Ownership of platform purchase-request processing state**
> **Status:** Proposed, unapproved. **Scope:** Platform request-processing state only, not final payment settlement.
> **Context:** Reporting and the request handler may disagree without defined authority.
> **Option A:** Single named writer with read-only consumers. **Benefit:** clear authority and conflict path. **Cost:** coordination or interface work.
> **Option B:** Multiple independent state writers. **Benefit:** potential local autonomy. **Cost:** conflict resolution, audit, and consistency risk.
> **Proposed decision:** A, pending domain/data approval.
> **Consequences:** writes go through an owned update boundary; reporting is a projection and must not rewrite request state; failure behavior must be tested.
> **Checks:** negative test for an unauthorized writer; simulate delayed Reporting and ensure the authoritative request state remains intact; verify audit trail.
> **Owners:** Transaction domain lead for data authority; engineering owner for mechanism; security owner for access review.
> **Revisit triggers:** domain authority changes; required cross-team update cases cannot use the boundary; test evidence shows the chosen owner becomes an unmanageable bottleneck.
> **Approval:** pending; no claim that this arrangement has been implemented.

**E10 — Trace checker and human audit.** Initial output is `{'R03': ['test']}`. Add R04 to the dictionary:

```
rows["R04"] = {"decision": "ADR-SEC?", "test": "T04", "risk": "RK05",
               "dependency": "DP06", "owner": ""}
assert missing_links(rows) == {"R03": ["test"], "R04": ["owner"]}
rows["R03"]["test"] = "T03"  # A reference only; not proof the test has passed
assert missing_links(rows) == {"R04": ["owner"]}
```

**Line by line:** First statement creates R04 with an empty owner. First assertion expects the two gaps. Next statement adds the *planned* T03 link. Final assertion confirms the structural gap is removed for R03 while R04 remains missing an owner. This code is an extension to the lesson's

program and must be executed in the same run after the original `rows` and `missing_links` definitions. An orphan `T99` cannot be detected because the program has **no test inventory** to search backward. Add a test registry and reverse links to check orphan artifacts. Even with a T03 link, R03 remains **blocked**: the freshness target, Reporting contract and durability tests are unapproved/unrun. Beware: Python can skip `assert` when run with optimization (`-O`); this exercise's assertions are learning checks, not production validation logic. [S14]

**E11 — Formal review.** Five example review participants and questions: product owner ("Which user outcome matters?"); transaction domain owner ("Which state am I accountable for?"); integration engineer ("How is a reporting update recovered after interruption?"); security/data owner ("What restricted data crosses each boundary?"); operations owner ("Which shared infrastructure fails and who restores it?"). An added finding: `F-06: Payment timeout may have caused an external action, but the local status is definitive`; classify as **Blocker for the state-changing path**, assign integration/transaction owners, and require a *failure simulation plus agreed external-contract handling*. A diagram correction can *show* an unknown outcome; only a designed, tested mechanism can *resolve* it.

**E12 — Readiness.** Example decision: `NOT READY` for state-changing integration; `CONDITIONALLY READY` only for synthetic-data discovery and non-production review artifacts. Three blockers: unapproved state owner, untested/reporting delivery mechanism, and unreviewed data access. Allowed: create contract questions and a synthetic failure test harness. Forbidden: production purchase execution and processing sensitive live records under the unreviewed flow. Required approvers: product/domain plus designated security/data and architecture authorities, as determined by actual organizational governance. Exit criteria: named state owner approved; required negative and interruption tests pass and are attached; Finance approves measurable freshness and verifies it using documented test evidence. Re-review after the missing evidence arrives.

## C. Model answers to practice questions (Q01–Q12)

| Question | Short model answer (add an example and trade-off when speaking) |
|---|---|
| **Q01** | A **concern** is the stakeholder's question; a **viewpoint** says how to frame that type of question; a **view** is the actual diagram or table. Multiple focused views avoid one unreadable drawing; keeping them aligned costs maintenance effort. |
| **Q02** | Context names the system boundary, users and direct outside systems. It omits code, internal classes and detailed runtime nodes. This speeds shared understanding but cannot settle internal ownership. |
| **Q03** | No: a component is a logical unit and does not by itself mean an independent service. H04 *proposes* a state repository/owner; approval is missing. A clear single writer reduces ambiguity at the cost of coordination. |

| Question | Short model answer (add an example and trade-off when speaking) |
|---|---|
| **Q04** | Data-flow tracks information, stores and boundary crossings; sequence tracks interaction order for a scenario. In this case, ask whether the Reporting payload is authorized and minimized. A high-level DFD alone cannot prove runtime ordering. |
| **Q05** | The caller knows the call timed out, not whether Payment acted. Mark the outcome uncertain and follow confirmed contract/reconciliation procedures. Showing false success is dangerous; simply retrying may also be dangerous. |
| **Q06** | Deployment places running instances and stores on infrastructure. Two application nodes can both depend on one unavailable database. Redundancy increases operating complexity and does not remove every failure point. |
| **Q07** | Compare labels, boundaries and ownership in all views; ask the approved system owner; log an open question and update the affected views/ADR. Never resolve a contradiction by guessing from one diagram. |
| **Q08** | Weights and scores make priorities visible; they are not test results. Access authorization and required correctness are gates. More delivery speed cannot justify prohibited data disclosure. |
| **Q09** | Use ID, status, context, alternatives, criteria, decision, positive/negative consequences, owner/approver, tests, dependencies, risks, history and observable reconsideration trigger. A new accepted ADR can supersede an old one without erasing history. |
| **Q10** | R03 → DFD/DEP → ADR-01 → T03 → RK03/DP03–05 → Finance owner; ADR-01 → R03 explains the backward reason. A `T03` cell is only a *planned link*, not a passing test or an approved target. |
| **Q11** | Record scope, reviewers, questions, evidence, findings, severity, owners, disposition and retest actions. Ask the alleged state owner to show approved write authority; a clean diagram cannot substitute for that evidence. |
| **Q12** | Conditional readiness permits a strictly limited scope while blockers remain. The architect cannot accept security, financial or compliance risk on behalf of owners lacking that authority. A narrower approval is not permission for production financial actions. |

## D. Cumulative exercise reference solution (K1–K5)

This is a **minimum acceptable content pattern**, not an already-completed learner submission.

- **K1 — Five views:** Recreate Context-01, Component-01, DFD-01, SEQ-01 and DEP-01 from Lessons 2–6 *without copying*, and add a one-line audience/question per view. The data-flow view must name data and stores; the deployment view must map logical application/database to runtime nodes.
- **K2 — Failure path:** Recreate SEQ-02 with a Payment timeout and **unknown** external outcome; do not infer a decline. Cross-check external Payment, transaction-state repository, and Reporting data in the other views. In the ten-minute spoken review, name one limitation of each view and one cross-view inconsistency you would test.
- **K3 — Comparison + ADR:** Use both reporting alternatives, agreed criteria, no invented proof, a conditional ADR-01 with benefits, costs, controls, owner, and a trigger such as an agreed freshness bound being breached. “Asynchronous always wins” earns no trade-off credit.
- **K4 — Trace:** Produce at least R01–R04 with explicit decision, diagram, test expected result, risk, dependency, owner, and **evidence status**. Flag missing approved target and unrun tests. Reverse-check ADR-01 from R03 and T03 from its requirement.
- **K5 — Review:** Create the REVIEW-01 finding log and a scoped disposition. The minimum defensible sample is **not ready for full integration**, with discovery-only work possibly allowed and evidence-based exit conditions. Severity and authority must be validated in an actual organization.

**Why this is senior rather than diagram copying:** Correctly refusing an unsupported claim, narrowing approval scope, linking an uncertainty to an owned test, and identifying when an ADR must be reconsidered are stronger evidence than drawing more boxes.

## E. Senior decision challenge — sample response and changed constraint (S1–S4)

**Sample spoken answer [HA + REC]:**

> “The business needs an employee to receive an honest processing status and Finance to see the related request later. Our diagrams do not identify who is allowed to write request state. I would not approve the full state-changing path yet. I would first add a small owner-and-writer annotation to the component view, plus an exception sequence for Payment timeout and local storage failure. If sensitive data crosses to Reporting, I would add a targeted data-flow excerpt showing fields and trust boundary. A new general diagram set would not resolve the main unknown. I would compare one authoritative platform-state writer with several independent writers, and propose one writer because it simplifies audit and conflict resolution, while accepting an extra integration and ownership boundary. I would write this as a proposed ADR, trace the state requirement to a storage-failure test, unauthorized-writer test, the external dependency and accountable owners. I would then ask the transaction/data owner to confirm authority, the integration team to provide contract and failure-test evidence, and the security owner to approve data movement. Until those gates pass, discovery may continue, but the financial request path is not ready to implement.”

**Seven required points mapped:**

| Point | Evidence in sample answer |
|---|---|
| Business goal | Honest employee status; dependable Finance visibility |
| Assumptions / missing facts | State writer/authority, provider outcome semantics, permitted reporting fields |
| Two options | One writer versus independent writers; conditional delivery scope |
| Decision and trade-off | Minimal component annotation + exception sequence; one writer costs coordination |
| Failure/security/data | Payment timeout, state-store failure, unauthorized update, sensitive reporting payload |
| Evidence/experiment | Contract, fault simulation, denied-writer test, review sign-offs |
| Owners/outcomes | Domain/data owner, integration engineer, security owner; go/no-go recorded when evidence arrives |

**S1:** A *small, targeted* component ownership annotation and exception sequence are initially sufficient for the missing owner/failure question. Add focused data-flow where the security concern requires it. A deployment check is required if the approval scope includes operating/runtime resilience, rather than assuming context alone proves it.

**S2:** Record the single-writer proposal in ADR-02. Compare its simpler authority and audit model with its coordination cost against independent writers' autonomy and conflict risk. Add triggers: rejected ownership sign-off, failed state-store tests, or a legitimate new cross-domain update need.

**S3:** Example trace: `R02 -> Component-01/SEQ-02 -> ADR-02 (Proposed) -> T02 (not run) -> RK02 -> DP02 -> Transaction domain owner`. In reverse, `ADR-02 -> R02` must be justifiable. The missing authority sign-off and test evidence are unresolved even if the table cells are populated.

**S4:** Full state-changing implementation: `NOT READY`. Narrow synthetic-data tests and requirements clarification: `CONDITIONALLY READY` with owners, limits and re-review. Only the actual authorized business, data, security and architectural decision roles can approve the relevant risks.

**Changed-constraint response:** If Finance demands a tighter approved freshness bound, revisit **ADR-01**, test alternatives against the new bound, and report the impact on delivery/operations. If Payment cannot confirm a status-query operation, remove any plan depending on that query; document the unknown-outcome handling requirement and request the real contract. This may increase the case for *deferring state-changing behavior* rather than making an unsafe retry assumption. Recheck the trace and review decision. **Do not claim the new requirement is achievable until measured.**

# 18. Assessment rubric, scorecard and completion rules

## A. Binding five-level rubric from the source [SR]

| Score | Evidence of capability |
|---|---|
| **1/5** | Cannot correctly explain the concept, even when prompted. |
| **2/5** | Knows words but needs help applying them; misses assumptions or risks. |
| **3/5** | Produces a workable answer; alternative comparisons or failure reasoning are weak. |
| **4/5** | Independently explains, applies, compares credible options, defends the trade-off, and identifies evidence or checks. |
| **5/5** | Does all of 4/5 and adapts to changed constraints, including second-order business, data, security, operations and delivery effects where relevant. |

**How to assess [SR + REC]:** Score **business framing, technical correctness, trade-offs, failure/security/data risk, evidence, execution/ownership and communication** when applicable. Each *applicable* senior-test dimension requires **at least 4/5**. A polished definition without a self-created artifact does not pass. The case's 5-minute target and example matrix scores are not assessment evidence.

## B. Checklist-specific completion gate

| ID | Explain without notes | Apply without notes | Defend without notes | Required evidence / status |
|---|---|---|---|---|
| **C019 Core** | Describe 5 distinct views, audiences, omissions | Produce 5 views + exception sequence; cross-check names and ownership | Select smallest extra views for a missing owner; explain limits | E01–07, Q01–07, K1–2, S1 — **Not yet assessed** |
| **C020 Core** | Explain options, criteria, ADR fields/triggers | Create scored or qualitative comparison + complete ADR | Change weights/constraint and defend the change | E08–09, Q08–09, K3, S2 — **Not yet assessed** |
| **C021 Core** | Explain forward and backward traceability | Link requirement → decision/view/test/risk/dependency/owner; label evidence | Find unsupported coverage/orphans and decide mitigation | E10, Q10, K4, S3 — **Not yet assessed** |
| **C022 Depth** | Explain review roles, findings and authority | Run a formal review and record scoped readiness | Defend gate, scope limits and accountable approvals | E11–12, Q11–12, K5, S4 — **Depth: not yet assessed** |

**C019 special check:** Do not pass it if even one of context, component, data-flow, sequence, or deployment views is missing. Do not substitute a sequence diagram for a data-flow diagram. **C020 special check:** no ADR passes without negative consequences **and** an observable reconsideration trigger. **C021 special check:** a test ID is not the same as a passing result. **C022 special check:** a meeting without a documented readiness decision does not pass.

## C. Copyable scoring sheet (leave blank until a real assessment)

| Dimension | Evidence observed (link or short note) | Score 1–5 | Revisit action / owner |
|---|---|---|---|
| Business framing | _____ | — | _____ |
| Technical correctness | _____ | — | _____ |
| Trade-offs | _____ | — | _____ |
| Failure/security/data risk | _____ | — | _____ |
| Evidence, verification plan | _____ | — | _____ |
| Execution and accountability | _____ | — | _____ |
| Clear professional communication | _____ | — | _____ |

| Checklist | Explain (1–5) | Apply (1–5) | Defend (1–5) | Result / retest need |
|---|---|---|---|---|
| C019 Core | — | — | — | Not assessed |
| C020 Core | — | — | — | Not assessed |
| C021 Core | — | — | — | Not assessed |

| Checklist | Explain (1–5) | Apply (1–5) | Defend (1–5) | Result / retest need |
|---|---|---|---|---|
| C022 Depth | — | — | — | Not assessed / may be deferred |

## D. Explicit F02 completion criteria [SR + REC]

1. Complete the **F01 prerequisite** separately; demonstrate PR-1–PR-4 before studying dependent diagrams. This reference does not certify F01.
2. For **each Core ID (C019, C020, C021)**, provide its own lesson exercises, Q evidence and K evidence. The learner must score **≥4/5 separately in Explain, Apply, and Defend**, without notes. A single nice portfolio does not replace individual ID checks.
3. Complete the **F02 running-case practice**: context diagram, transaction sequence, ADR comparing two realistic options, and **ten-minute** verbal explanation. This guide also requires the other three views because C019 explicitly includes them.
4. Pass the **senior decision test** (§16.C), answering all seven items and scoring **≥4/5 on every applicable rubric dimension**. Show the smallest additional views/tests, a documented decision, and accountable owners with measurable gates.
5. For **C022 (Depth)**, keep a **separate** explain/apply/defend score and review disposition. It may be deferred under v3's Core/Depth rule, but must not be shown as passed if untested. If attempted, use the same ≥4/5 requirement.
6. Revisit any dimension below 4/5; document a weakness, correction, new evidence and retest. Mark an item complete **only after demonstrated performance**, never after reading this file.

**Outcome labels:** `Core pass`, `Core not yet passed`, `Depth pass`, `Depth deferred/not yet assessed`, and `Needs reassessment after changed constraint`. An F02 *Core pass* does not imply knowledge of later T/D/E cards or actual enterprise implementation authority.

# 19. Primary-source register and verification limits

**Verification date: 2026-10-24.** The items below were checked against their **primary issuing organization** or creator-maintained documentation. These references support the named *external facts* only; they do not turn our recommended diagrams or hypothetical business details into official mandates.

| Key | Official/primary reference (checked 2026-10-24) | What it verifies — and what it does not |
|---|---|---|
| **S1** | ISO — ISO/IEC/IEEE 42010:2022, architecture description | Scope of architecture descriptions and viewpoints; its public abstract expressly does **not** prescribe diagramming method, notation or tool. **Limit:** full paid standard normative text was not inspected. |
| **S2** | C4 model — creator's official site | Layered system context/container/component and supporting dynamic/deployment diagrams; notation/tool independence. Not a rule that every team must draw every level. |
| **S3** | C4 — system context | Central system, surrounding people and external systems, big-picture audience. |
| **S4** | C4 — component | A strict C4 component diagram zooms inside a single container. Our generic logical component view is simplified, not a claim of formal C4 conformance. |
| **S5** | Microsoft — Threat Modeling Tool / data-flow diagram | Data flows and trust boundaries in threat modeling; does not prove security controls exist in the fictional case. |
| **S6** | Object Management Group — UML 2.5.1 specification | UML is issued by OMG. ASCII sequence illustration is intentionally informal, not a strict UML artifact. |
| **S7** | C4 — dynamic diagram | Runtime collaboration; communication/sequence-style alternatives for a use case. |
| **S8** | C4 — deployment diagram | Software/runtime instances mapped to deployment nodes; optional infrastructure dependencies. |

| Key | Official/primary reference (checked 2026-10-24) | What it verifies — and what it does not |
|---|---|---|
| **S9** | Carnegie Mellon SEI — Architecture Tradeoff Analysis Method | Evaluating architecture against competing business/quality goals and finding risks/trade-offs. This guide does **not** claim to execute full ATAM. |
| **S10** | AWS — Architectural decision record process | ADR context, decision, consequences, lifecycle and supersession. Vendor guidance, not alleged enterprise policy. |
| **S11** | AWS — ADR best practices | Preserve history, record owner, keep superseded records and shared decision log. |
| **S12** | NASA — Systems Engineering Handbook Appendix D, Requirements Verification Matrix | Requirement identifiers, source and planned verification in an example matrix. This guide's extra risk/owner/decision columns are its own teaching format. |
| **S13** | NASA Software Engineering Handbook — SWE-059 bidirectional requirements/architecture traceability | NASA's own traceability practice. It is **not** a compliance obligation for a commercial platform. |
| **S14** | Python language tutorial — data structures and Python language reference — `assert` | Built-in lists/dictionaries and assertion semantics. `assert` can be disabled with optimization; do not use it as the sole production control. |
| **S15** | AWS Well-Architected Framework — review process | Reviews should uncover critical issues, be constructive, and lead to prioritized action. Not this hypothetical organization's approval process. |
| **S16** | Microsoft Azure Well-Architected — Architecture design diagrams | Distinct purposes of context, component, data-flow, sequence and deployment views; audience-driven choice and cross-view clarity. |

**Interpretation rules:** ISO is the normative issuer for its standard; only its **public description** was read. C4 is the creator's published convention. OMG defines UML, which the guide does not claim to reproduce precisely. NASA, Microsoft, AWS and SEI give documented practices in their own contexts. **None** certifies the proposed Spending Platform or the exact requirements of a private hiring team. All invented assumptions have the `[HA]` label and H-IDs; all instructional conventions use `[REC]`.

### External facts still requiring case-specific confirmation

- An **official job requisition and current interview process** have not been provided or established; role-specific internal tools, standards, architecture style and approval gates remain **unverified**.
- The actual **payment and risk interfaces** (including outcome lookup, timeout behavior and authority for final payment states) are unknown. The guide uses placeholders, not claimed vendor capabilities.
- Real **data classification, lawful processing, authorization, operating targets, durability, redundancy, release scope and finance freshness target** all require authorized subject-matter owners and evidence.
- The sample scoring weights, testing criteria, node names, fictional responsibilities and draft statuses are **teaching assumptions**, not measured outcomes or signed decisions.

# 20. Final completeness audit — document coverage versus learner mastery

This is an **authoring audit**, not a claim that anyone has passed an assessment. Its scope is the authoritative **F02 (C019–C022)** card only. Requirements from later cards were not imported.

| Requirement | Audit result at document creation | Where verified in this guide | Outstanding limitation |
|---|---|---|---|
| **Master scope: C019 Core** | **Included**: 5 separate views, audiences, assumptions and cross-view review | L01–L07, E01–E07, Q01–Q07, K1–K2, S1 | Learner has not produced and defended their own diagrams |
| **Master scope: C020 Core** | **Included**: explicit options, criteria, invented score math, complete ADR, consequences and revisit triggers | L08–L09, E08–E09, Q08–Q09, K3, S2 | No organizational acceptance or measured evidence |
| **Master scope: C021 Core** | **Included**: requirements → decisions/views/tests/risks/dependencies/owners, forward and reverse checks | L10, E10, Q10, K4, S3 | Sample test links are not passed tests; real owner sign-off missing |
| **Master scope: C022 Depth** | **Included and separately labeled**: formal review simulation, readiness criteria and documented disposition | L11–L12, E11–E12, Q11–Q12, K5, S4 | Depth may be deferred; learner not scored |
| F01 prerequisite taught before F02 dependencies | **Bridge and check included** | §2, PR-1–PR-4 | This does **not** certify completion of the separate F01 card |

| Requirement | Audit result at document creation | Where verified in this guide | Outstanding limitation |
|---|---|---|---|
| Purpose, terms, example, exercise, failure/trade-off, interview phrasing for each checklist item | **Included** (with C019 split into focused diagram lessons) | L01–L12; interview follow-up at every lesson | Human feedback on actual answers can occur only when learning starts |
| Practical and failure examples; senior reasoning | **Included** | SEQ-01/02, DFD, DEP, ADR, review findings, §16.C | Mechanisms involving later cards are explicitly left at boundary level |
| Executable Python + line-by-line explanation | **Included** | L10, E10 and §17.B | Code checks row completeness only; code has not demonstrated real integration correctness |
| Practice questions, cumulative exercise, senior test, separate answer key | **Included** | §16 and separate §17 | No real learner answers or live interview results |
| Source fact/assumption separation and primary references | **Included**: key [VF/SR/HA/REC] and source register | §1, H01–H09, §19 | Full ISO standard text and organization-specific facts are **not** verified |
| Scoring rubric and pass/retake gates | **Included**: 1–5 per dimension; Core/Depth separate; ≥4/5 gates | §18 | Scores left blank until demonstrated evidence exists |
| A scope-respecting, static reference file | **Included** | This single Markdown document | No interactive teaching or assessment has been conducted |

**Checklist-by-ID audit:** `C019 = present; C020 = present; C021 = present; C022 = present (Depth)`. **Missing F02 checklist items: none.** This means *the document contains their teaching and assessment paths*, **not** that mastery is demonstrated.

**Verified externally:** ISO public abstract; creator-maintained C4 diagram conventions; OMG specification listing; official Microsoft data-flow/architecture guidance; NASA traceability guidance; SEI ATAM description; AWS ADR/review guidance; Python language documentation. The linked primary references above were checked on **2026-10-24**. **Unverified:** official vacancy specifics, internal architecture/governance, provider contracts, signed business targets, and any actual design/test/approval evidence.

**Final acceptance status: Learning-guide deliverables complete; learner completion not assessed.** Before declaring the *card* complete, collect the independent C019–C021 exercise/portfolio/senior-test evidence, grade applicable dimensions at ≥4/5, log weaknesses and retest. Record C022 separately as `Depth pass` or `Depth deferred/not yet assessed`. No fictional result should be presented as personal experience.